European Network of
Transmission System Operators
For Electricity

Administration Guide

Energy Communication Platform
Version 4.16.0

Publication Date: 06.05.2025

Energy Communication Platform, version 4.16.0

This software is protected by copyright.

Copyright © 2025, ENTSO-E AISBL
All rights reserved.

This software was developed for ENTSO-E by Unicorn Systems a.s.
www.unicorn.com

Licenses and third-party components:

- This product includes software developed by the Apache Software Foundation (https://www.apache.org/)
- This product includes following third-party open-source components:
  - Apache Tomcat (https://tomcat.apache.org/)
  - Spring Boot (https://spring.io/projects/spring-boot/)
  - ActiveMQ Artemis (https://activemq.apache.org/components/artemis/)
  - Qpid JMS (https://qpid.apache.org/components/jms/)
  - Apache Camel (https://camel.apache.org/)
  - Logback (https://logback.qos.ch/)
  - Spring Framework (https://spring.io/projects/spring-framework/)
  - Hibernate (https://www.hibernate.org/), licensed under LGPL 2.1
  - AngularJS (https://angularjs.org/)
  - Liquibase (https://www.liquibase.org/)

1. Contents

1. Contents ... 3
2. Executive Summary ... 9
3. Revision History ... 10
4. Common Topics ... 12
4.1. How to change the configuration ... 12
4.1.1. System configuration ... 12
4.1.2. Runtime configuration ... 12
4.2. How to recover from an incorrect configuration ... 14
4.3. How to set a different application theme ... 14
4.4. How to set Project and Environment name ... 15
4.5. Recommended JMX remote settings ... 15
4.5.1. JMX authentication settings ... 16
4.5.2. JMX SSL configuration settings ... 17
4.5.3. JMX monitoring with JConsole ... 17
4.6. Multiple Root Certificate Authorities ... 17
4.7. Encrypted passwords ... 18
4.7.1. How to use Jasypt ... 19
4.8. How to enable separate Audit Log file ... 19
4.9. How to send logs to Syslog server ... 20
4.9.1. Filter ERROR log messages ... 21
4.10. How to configure HSM ... 21
4.11. Cleaning job for expired certificates ... 23
4.12. How to configure Keycloak integration with LDAP ... 24
4.13. How to configure LDAP authentication ... 24
4.14. How to integrate ECP into existing ELK stack ... 27
4.14.1. Prerequisites ... 27
4.14.2. ELK configuration process ... 27
4.15. How to log client IP behind proxy ... 28
4.16. Compression job for Derby or PostgreSQL database ... 29
4.17. How to enable auditing of the configuration files ... 29

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

4.18. How to configure a maximum request size ... 30
4.19. How to enable full stack trace logging ... 31
4.20. Database backup and restore ... 31
4.21. How to log to the console ... 33
5. ECP Endpoint Administration ... 34
5.1. User authentication ... 34
5.1.1. GUI Authentication ... 34
5.1.2. Web Services Authentication ... 35
5.1.3. Monitoring Authentication ... 35
5.1.4. Passwords in Hash ... 35
5.2. How to configure unsuccessful login attempts ... 36
5.3. How to configure message paths ... 36
5.3.1. How to add a new message path ... 36
5.3.2. Import message paths in a file ... 37
5.3.3. How to configure processing of centrally managed message paths ... 39
5.3.4. Message path selection ... 39
5.4. How to configure FSSF ... 40
5.4.1. Sending messages through FSSF ... 40
5.4.2. Receiving messages through FSSF ... 41
5.4.3. Message type matching rules ... 42
5.5. How to configure AMQP API ... 43
5.5.1. Setup of multiple receive queues ... 43
5.5.2. How to enable AMQP send handler ... 44
5.5.3. How to secure AMQP API ... 44
5.5.4. How to disable AMQPS ... 45
5.6. How to configure archiving ... 46
5.6.1. Setup of file system archiving ... 46
5.6.2. Registration of custom archive handler ... 47
5.7. How to configure receive handlers ... 48
5.8. How to configure send handlers ... 48
5.9. How to configure message expiration ... 49
5.9.1. TTL per Message Type ... 50
5.10. How to configure message priority ... 50

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

5.11. How to configure logging ... 51
5.11.1. Configure log file location ... 51
5.11.2. Configure logger severity ... 51
5.11.3. Configure log size and rotation ... 51
5.11.4. Log full package names ... 52
5.12. Configure HTTPS ... 53
5.12.1. Tomcat configuration ... 53
5.13. How to enable SOCKS Proxy ... 54
5.14. How to add plugins to ECP ... 55
5.15. How to enable direct communication ... 55
5.16. How to configure message deleting ... 56
5.17. How to configure Component Directory synchronization ... 56
5.18. How to configure message signing ... 57
5.18.1. How to configure multiple message signing ... 57
5.19. How to configure NAT ... 58
5.20. How to configure antivirus ... 59
5.21. How to restrict Jolokia access ... 60
5.22. How to enable JMX for remote monitoring ... 60
5.22.1. Enabling JMX ... 60
5.23. How to restore registration process ... 61
5.24. External database configuration ... 61
5.25. Other configuration settings ... 62
5.25.1. How to configure connectivity check ... 63
5.25.2. How to configure message compression ... 63
5.25.3. How to configure JMS headers validation ... 64
5.25.4. How to configure message table compression job ... 64
5.25.5. How to configure connection URL parameters of ECP Brokers ... 65
5.26. Certificates renewal ... 66
5.27. Backup and restore data and configuration ... 67
5.27.1. Windows installation ... 67
5.27.2. Linux installation ... 67
5.28. Configure cluster lock intervals ... 68
5.29. IP change of the server ... 68

5.30. Component revocation ... 68
5.31. Multiple ECP Endpoints with one Internal Broker ... 69
6. Internal Broker configuration ... 70
6.1. Enable HTTPS ... 70
6.2. Keystore configuration ... 70
6.3. Enabling remote JMX access ... 71
6.4. Passwords encryption ... 72
6.5. User management ... 73
6.6. LDAP authentication ... 73
6.7. How to enable direct communication ... 77
6.8. How to connect BA over AMQPS ... 78
6.9. How to disable AMQPS and authentication ... 79
6.10. Plugins properties ... 79
6.10.1. EccospAuditLoggerPlugin ... 79
6.10.2. EcpBaseDirectPlugin ... 80
6.10.3. EcpAuthorizationDirectPlugin ... 80
6.11. Plugins user credentials ... 81
6.12. Backup and restore data and configuration ... 82
6.12.1. Windows installation ... 82
6.12.2. Linux installation ... 82
7. ECP Component Directory Administration ... 84
7.1. User authentication ... 84
7.1.1. Monitoring Authentication ... 84
7.1.2. Passwords in Hash ... 85
7.2. How to configure unsuccessful login attempts ... 85
7.3. HTTPS connector configuration ... 86
7.4. Certificates renewal ... 86
7.4.1. Certificate Authorities Update ... 87
7.5. How to enable JMX for remote monitoring ... 87
7.5.1. Enabling JMX ... 88
7.6. Central Message Paths Management ... 88
7.6.1. How to add a new centrally managed message path ... 88
7.7. Central Management of TTL per Message Type ... 90

7.7.1. Message Paths Conflict Resolution ... 90
7.8. Registration keystores with specific component code ... 91
7.9. How to configure synchronization with remote Component Directories ... 92
7.9.1. Group of ECP networks ... 93
7.9.2. Configuration of synchronization via GUI ... 95
7.10. Cleaning job for expired specific registration certificates ... 95
7.11. How to restrict GUI access ... 95
7.12. How to configure components expiration ... 96
7.13. External database configuration ... 96
7.14. Another configuration ... 98
7.15. Backup and restore data and configuration ... 99
7.16. How to create CA keystore ... 99
8. ECP Broker Administration ... 101
8.1. How to change broker configuration ... 101
8.2. How to configure component filter ... 101
8.3. How to configure NAT ... 102
8.4. Certificates renewal ... 102
8.5. Artemis configuration ... 103
8.5.1. Enable HTTPS ... 103
8.5.2. Enabling remote JMX access ... 104
8.5.3. Passwords encryption ... 104
8.5.4. User management ... 105
8.5.5. LDAP authentication ... 105
8.6. Plugins properties ... 109
8.6.1. EcpPrometheusMetricsPlugin ... 109
8.6.2. EcpBasePlugin ... 110
8.6.3. EcpAuthorizationPlugin ... 110
8.7. Plugins user credentials ... 112
8.8. Backup and restore data and configuration ... 112
8.9. Required Broker configuration ... 113
9. Monitoring ... 115
9.1. Endpoint ... 115
9.2. Broker and Internal Broker ... 117

9.3. Component Directory 117
9.4. Info Endpoint 118
9.5. Health Endpoint 118
9.6. Readiness Endpoint 121
9.7. Prometheus Monitoring 121
9.7.1. Component Directory and Endpoint 121
9.7.2. Broker and Internal Broker 124
10. Figure Content 126

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

2. Executive Summary

This document describes administrative tasks for all ECP components.

Please NOTE: Every ECP user needs to contact the project leader to get the integration document specific for each project.

Please NOTE: If you use the ECP Endpoint as a stand-alone application, there is only one mandatory chapter – *How to configure message paths.*

This document is targeted at Administrators and Business Users.

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

3. Revision History

Version	Date	Author	Description
4.1.4	02.02.2018	Laura Betcherová	Documentation review
4.2.0	21.03.2018	Josef Brož	Version update
4.3.0	26.06.2018	Josef Brož	Version update
4.3.0	05.10.2018	Barbora Klabačková	Java update procedure
4.3.0	18.10.2018	Barbora Klabačková	Certificate renewal update
4.4.0	04.02.2019	Jiří Kakrda	Broker NAT
4.4.0	05.06.2019	Barbora Kánská	Document revision
4.5.0	22.08.2019	Jiří Kakrda	HSM support
4.6.0	05.11.2019	Jiří Kakrda	Version update
4.6.1	18.03.2020	Anna Langrová	Version update
4.7.0	28.06.2020	Lukáš Voříšek, Jiří Kakrda, Lukáš Rezner	CD Synchronization, Hash passwords, Prometheus monitoring, CD Restrict GUI access
4.7.1.	31.07.2020	Anna Langrová, Jiří Kakrda, Lukáš Voříšek	Database compression job, Proxy IP, AMQPS disable
4.7.2	18.11.2020	Jiří Kakrda, Lukáš Voříšek	CD Synchronization details, CDS
4.8.0	12.04.2021	Jiří Kakrda, Tomáš Vondra	TLS v1.3, disable internal broker, configure unsuccessful login attempts
4.8.1	09.07.2021	Lukáš Voříšek	Auditing description
4.8.2	03.12.2021	Daniel Janota, Jiří Kakrda	App theme validation, Project and Environment names, FSSF default extension, DLQ, ActiveMQ configuration files, Message table compression, restricted access error page, remote CD removal, formatting

4.9.0	26.05.2022	Jiří Kakrda	ECP 3 *secondaryUrl* property, TTL explanation, LDAP authentication, Prometheus metrics update, synchronization job update, certificate renewal update, info endpoint, message path selection
4.10.0	07.11.2022	Daniel Janota, Jiří Kakrda	Connectivity check default value, Hibernate, compression job update, Project and Environment names update, Health and Readiness endpoints, editing runtime configuration in GUI, JMX authentication update
4.11.0	16.05.2023	Jiří Kakrda	Extension of connection over AMQPS, ECP 3 removal, Keycloak role mapping, Prometheus Broker metrics, New metrics and new property, FSSF extension property, Certificates delay property, Hawtio permissions, LDAP role properties, Keycloak and LDAP WS authentication
4.12.0	27.10.2023	Jiří Kakrda	Import message path timezone, removal info about CD restart after renewal, new Prometheus metrics, message priority property, full stack trace logging, Component revocation chapter, health endpoint metrics check
4.14.0	10.06.2024	Jiří Kakrda	Artemis, CSRF secret property, CD and Broker Windows removal, Keycloak LDAP integration, queue prefix, *brokerConnectionDelayHours* property, Endpoint chapter Certificates renewal update, new Broker Prometheus metric
4.15.0	21.10.2024	Jiří Kakrda	New chapters *Database backup and restore, Plugins properties, Plugins user credentials,* new *baQueuePrefix* property
4.16.0	06.05.2025	Jiří Kakrda	Broker URL connection parameters, Hawtio removal, new chapters *How to log to the console* and *Monitoring Authentication*, new LDAP properties, DB restore manual push info, AMQPS chapters extended

4. Common Topics

4.1. How to change the configuration

ECP has two sources of configuration:

- *System configuration*
- *Runtime configuration*

4.1.1. System configuration

System configuration is in the properties file on the disk. After changing the properties, the component must be restarted to update the configuration. System configuration contains only parameters that are required for start-up (such as database connection parameters). System configuration must be manually propagated to all nodes of the cluster.

Location of the system configuration files for all ECP components:

Component	Linux	Windows	Description
ECP Endpoint	/etc/ecp-endpoint/ecp.properties	<installation directory>/ecp.properties	System configuration of ECP Endpoint.
ECP Component Directory	/etc/ecp-directory/ecp-directory.properties	-	System configuration of ECP Component Directory.
ECP Broker	/opt/ecp-broker/config/broker.properties	-	System configuration of ECP Broker.

4.1.2. Runtime configuration

Runtime configuration can be changed during the operation of the system without the need to restart it. When using HA deployment, configuration changes will affect all nodes in the cluster.

The runtime configuration is available only on the Endpoint and Component Directory.

4.1.2.1. Editing the configuration in the web interface

Open the Component Directory or Endpoint web interface and navigate to the Settings page. Scroll down to the Runtime Configuration section and click the Edit Configuration button. A new page will open with all runtime properties. Each property can be edited after clicking on the edit button or set to a default value by clicking on the refresh button. Properties with an index can be added or removed by clicking on the plus or minus buttons.

After all desired configuration changes have been performed, click on the Save button. Then the configuration changes will be uploaded to the server and a validation will be performed.

When a validation is successful, Component Directory/Endpoint will reload its configuration shortly. Otherwise, an error message will be displayed with a list of invalid properties. Invalid properties will also be highlighted with a red colour in the table.

4.1.2.2. Export current configuration

Alternatively, it is possible to export the configuration to a properties file. Open the Component Directory or Endpoint web interface and navigate to the Settings page. Scroll down to the Runtime Configuration section and click the Export Configuration button. A properties file with the current configuration will be downloaded and will have the following file name pattern: ***<Component Code>-configuration.properties***.

4.1.2.3. Editing the configuration

Open the properties file with a text editor. Any text editor will work although an editor with syntax highlighting (Atom, Notepad++, etc.) is recommended for better readability and navigation.

Each property entry has its purpose comments (example values, possible enum values, further description etc.). Lines starting with # are ignored by the interpreter while importing configuration back into system.

Note: the single backslash character is considered as an escape sequence, therefore, if backslash is required it should be presented twice e.g. *dataDirectory=C:\\ECPEndpoint*.

There is a limit of 1000 items in list-type property (property, which include index [n]).

4.1.2.4. Import the updated configuration

Change desired configuration properties and upload the configuration back into the system. The Import Configuration button is beside the Export Configuration button.

The result of configuration import will be shown at the top of the page.

There are two possible results:

- *Import was successful.*

> *This indicates successful processing, and your Component Directory/Endpoint will reload its configuration shortly.*

- *Import was not successful.*

> *This message will indicate what went wrong e.g. unknown configuration property, invalid value, etc. All invalid properties will be present in the error message to assist the administrator in resolving the issues.*

4.2. How to recover from an incorrect configuration

If the application fails to start due to an incorrect configuration, it is possible to use recovery mode. Starting in recovery mode means that the application will load only minimal configuration to display visual use cases to allow the import and export of the current configuration and resolve the issue.

Recovery mode is activated in the system configuration file. To enable recovery mode, use the following property: *recoveryMode=true*. This property needs to be set in the system configuration.

A typical recovery procedure consists of the following steps:

> *Stop the failing application*

> *Locate the system configuration file and paste "recoveryMode=true" to the end of the file (without quotes)*

> *Start the application and change the configuration using the runtime configuration method*

> *Stop the application and either remove or comment out the line with the recovery property*

> *Start the application*

4.3. How to set a different application theme

ECP allows the theme of the application to be changed to suit distinct environments (production, test, etc.). This property can be changed in the runtime configuration.

Property	Description	Default value
ecp.appTheme	This property allows customization of the look and feel of the ECP application. Permitted values are DEFAULT, BLUE, WHITE. Default value will be used in	DEFAULT

case that a value is different than permitted values.

4.4. How to set Project and Environment name

During the registration process of the ECP Component Directory and Endpoint, project and environment names can be defined. These names are then displayed in the ECP header and browser tab name in the following pattern: Component Directory / Endpoint | Project | Environment.

Administrator can fill both names, one of them, or none. Maximum length is 16 characters and names must contain only alphanumeric character and a dash character.

Names can later be changed on *Settings* page, in section *Names*, or in the runtime configuration with these properties:

Property	Description	Default value
ecp.projectName	Project name. If the value is invalid, name will be empty.	empty
ecp.envName	Environment name. If the value is invalid, name will be empty.	empty

These names can be used for easier recognition of the ECP component.

The project name can then specify the project to which component belongs, for example, CORE or Transparency.

Environment name can then specify in which environment the component is running, for example, PROD or ACCE.

4.5. Recommended JMX remote settings

All ECP components supports monitoring by using JMX. To activate this function, see the JMX chapter for a given component. This chapter describes how to configure a secured remote JMX access for ECP Component Directory and for ECP Endpoint.

When enabled, Remote JMX interface should be secured by authentication (login/password) and SSL. The following JMX remote properties must be set:

- *com.sun.management.jmxremote=true*
- *com.sun.management.jmxremote.port=<PORT_NUMBER>*

- *com.sun.management.jmxremote.rmi.port=<PORT_NUMBER>*
- *com.sun.management.jmxremote.password.file=<PATH_TO>/jmxremote.password*
- *com.sun.management.jmxremote.access.file=<PATH_TO>/jmxremote.access*
- *com.sun.management.jmxremote.ssl=true*
- *com.sun.management.jmxremote.registry.ssl=true*
- *com.sun.management.jmxremote.ssl.config.file=<PATH_TO>/jmxremote.ssl*

4.5.1. JMX authentication settings

JMX authentication is based on predefined users and their permissions. Users and their passwords are specified in ***jmxremote.password*** file. User permissions are specified in ***jmxremote.access*** file. Those files must be created and referenced by *com.sun.management.jmxremote.password.file*

and *com.sun.management.jmxremote.access.file jmx* property.

- *Example of jmxremote.password file content:*
  - *User user_password*
  - *Administrator admin_password*
- *Example of jmxremote.access file content:*
  - *User readonly*
  - *Admininstrator readwrite*

Security policy of JMX requires restricted access to the ***jmxremote.password*** file. The file must be readable and writeable by the owner only. In addition to this, file must be readable and writeable only by the user who starts the ECP server.

To setup required permission on Linux machine, use the following commands:

- *chmod 600 jmxremote.password*
- *chown <user>:<group> jmxremote.password*
  - replace <user> and <group> with ecp user and group

To setup required permission on Windows machine, use the following commands. If ECP is not installed as a service, replace SYSTEM with a user who runs ECP:

- *icacls jmxremote.password /grant: SYSTEM:(R,W)*
- *icacls jmxremote.password /inheritance:r*
- *Use Windows Explorer to navigate to the jmxremote.properties file - show file properties – tab security – click advance button and change owner to SYSTEM and confirm dialog.*

4.5.2. JMX SSL configuration settings

Server keypair/certificate must be created to enable remote JMX SSL. OpenSSL tool can be used to achieve this with self-signed certificates.

JMX SSL settings file ***jmxremote.ssl*** must be created and referenced by *com.sun.management.jmxremote.ssl.config.file* property.

Example of *jmxremote.ssl* file:

- *javax.net.ssl.keyStore=/path/to/keystore.jks*
- *javax.net.ssl.keyStorePassword=keystore_password*
- *javax.net.ssl.trustStore=/path/to/truststore.jks*
- *javax.net.ssl.trustStorePassword=truststore_password*

Keystore path is specified by the *javax.net.ssl.keyStore* property. Java Keystore with private key matching the server name along with certificate chain is expected.

Truststore is specified by the *javax.net.ssl.trustStore* property. Java Keystore with trusted certificates is expected.

4.5.3. JMX monitoring with JConsole

If you use JConsole for JMX monitoring, it must be started with parameters pointing to a truststore. Truststore must contain a certificate signed by the same CA as certificates in ECP.

```
jconsole -J-Djavax.net.ssl.trustStore=<path-to-truststore>/truststore.jks
         -J-Djavax.net.ssl.trustStorePassword=<truststore-password>
```

4.6. Multiple Root Certificate Authorities

Every ECP Component trusts the Root Certificate Authority (CA) for the authentication purposes. Root CA issues the certificates for the Integrated CA of every Component Directory. Every authentication certificate in the ECP network is issued by an Integrated CA. ECP components, which authentication certificates come from different Root CA, cannot communicate to each other by default.

Component Directory administrator have possibility to import Root CA and Integrated CA from another Component Directory to its own Component Directory in order to enable communication between the components with different Root CA in one ECP network. Certificate Authorities from another Component Directory are distributed to Endpoints and Brokers by synchronization jobs. To enable communication between directories, Administrator of the second CD will have to import CA from the first CD.

Table “Certificates” on Settings page on Component Directory and Endpoint is divided into two tabs – “Local” and “Network”. “Local” tab shows own certificates or certificates belonging to the home Component Directory. “Network” tab show certificates belonging to another Component Directories.

Component Directory Administrator can import Root CA (Global CA) and Integrated CA of another Component Directory by clicking on “Import Network Certificate” button on Settings page. It is possible to import multiple different certificates of the same type belonging to one Component Directory. During import, Administrator can manually write component code of another Component Directory, or it is possible to read common name of imported Integrated CA and use it as a component code. Administrator must check that common name from a certificate is same as a component code of another Component Directory. If common name and code are different, Administrator must write component code manually.

After Certificate Authorities update, Component Directory and Broker must be manually restarted in order to reload SSL context. Endpoint will restart its external broker automatically.

4.7. Encrypted passwords

For security reasons, passwords in properties files can be written in an encrypted form instead in a plain text. Encrypted password in the properties file must be wrapped in brackets with the keyword ENC.

Following properties must be set to enable passwords decryption in this location:

- *Linux system: /etc/systemd/system/<ecp-directory.service.d or ecp-endpoint.service.d>/env.conf*
- *Windows system: Java Options in Java tab (command <install_path>/tomcat/bin/tomcat10w.exe //ES/ecp-endpoint)*

Property	Description	Default value
ecp.password.location	Path to the file with stored password, which was used for encryption. Password in the file must be in the first line of the file.	N/A

jasypt.encryptor.algorithm	Algorithm used for encryption. All passwords must be encrypted with this algorithm.	PBEWITHHMACSHA512ANDAES_256

4.7.1. How to use Jasypt

Jasypt CLI tools can be used for a password encryption: http://www.jasypt.org/cli.html.

Following steps are requested:

1. *Create a new file with the required encryption key. Save the file as the **ecp-password.properties** to the existing installation folder of ECP Endpoint.*
2. *Access the Jasypt folder.*
3. *Windows: Run ./encrypt.bat input=YourPassword password=EncryptionKey algorithm=PBEWITHHMACSHA512ANDAES_256 ivGeneratorClassName=org.jasypt.iv.RandomIvGenerator*

   Linux: Run *./encrypt.sh input*=YourPassword *password*=EncryptionKey *algorithm=PBEWITHHMACSHA512ANDAES_256 ivGeneratorClassName=org.jasypt.iv.RandomIvGenerator*
4. *Save your encrypted password to the demanded configuration file.*

   Example of an encrypted password in ***ecp-users.properties*** file:

   > *ecp.endpoint.users[0].password = ENC(XcBjfjDDjxeyFBoaEPhG14wEzc6Ja+Xx+hNPrJyQT88=)*
5. *Restart ECP component.*

4.8. How to enable separate Audit Log file

By default, all audit information is logged at the INFO level into the ECP application log file. Audit log records can be also written to separate file only. Rolling over policy based on file size is then applied for audit log file. Separate audit log file can be enabled by the following settings:

- *Add ECP profile ecp-audit to spring.profiles.active property in properties file*
- *Set the path to the log directory via the logging.path system configuration property (only when upgrading from ECP version 4.3.)*
- *Restart ECP component*

List of Audit events and for which components they are relevant is documented in the ECP AuditLog excel file.

4.9. How to send logs to Syslog server

It is possible to send ECP log messages to the Syslog server by adding Syslog appender configuration in the ecp-logback.xml file. Path to the ecp-logback.xml is defined in *logging.config* property. This file is in the ECP installation directory by default.

Example of the Syslog appender configuration:

```
<appender name="SYSLOG" class="ch.qos.logback.classic.net.SyslogAppender">
    <syslogHost>localhost</syslogHost>
    <facility>LOCAL0</facility>
    <suffixPattern>[%thread] %logger %msg</suffixPattern>
</appender>
```

Change *syslogHost* value to the IP of the Syslog server. You can also change *facility* and *suffixPattern* values if required.

Place Syslog appender configuration inside one of the *<springProfile>* sections, which is used by ECP. *<springProfile>* section is used, if name attribute in *<springProfile>* section matches any profile defined in *spring.profile.active* in ecp/ecp-directory.properties configuration file. If name attribute starts with "!" character, it means that *<springProfile>* section will be used only if *spring.profile.active* does not contain this profile. If you use default profiles in *spring.profile.active* property, *<springProfile name="!console-logging">* section is recommended to place Syslog appender configuration.

Next, add *<appender-ref ref="SYSLOG"/>* to the *<root level="INFO">* section.

Example of Syslog appender configuration defined in *<springProfile name="!console-logging">* section:

```
<springProfile name="!console-logging">
    <appender name="SYSLOG" class="ch.qos.logback.classic.net.SyslogAppender">
        <syslogHost>localhost</syslogHost>
        <facility>LOCAL0</facility>
        <suffixPattern>[%thread] %logger %msg</suffixPattern>
    </appender>
    <root level="INFO">
        <appender-ref ref="FILE"/>
        <appender-ref ref="SYSLOG"/>
    </root>
</springProfile>
```

Finally, save ecp-logback.xml and restart ECP component.

4.9.1. Filter ERROR log messages

If you wish to send only ERROR level messages, you can add filter into the Syslog appender configuration:

```
<appender name="SYSLOG" class="ch.qos.logback.classic.net.SyslogAppender">
    <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
        <level>ERROR</level>
    </filter>
    <syslogHost>localhost</syslogHost>
    <facility>LOCAL0</facility>
    <suffixPattern>[%thread] %logger %msg</suffixPattern>
</appender>
```

ERORR messages can also be sent to the ECP stderr log file by adding this appender with <appender-ref ref="STDERR"/> reference:

```
<appender name="STDERR" class="ch.qos.logback.core.ConsoleAppender">
    <target>System.err</target>
    <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
        <level>ERROR</level>
    </filter>
    <encoder>
        <pattern>${FILE_LOG_PATTERN}</pattern>
    </encoder>
</appender>
```

4.10. How to configure HSM

Before configuring ECP to use HSM (Hardware Security Module), make sure your HSM is running, configured properly and with initialized token and slot. Your HSM must be compatible with PKCS 11 standard. Each ECP component requires its own slot in HSM.

HSM can be used only with clean, unregistered ECP components. Migration of existing ECP components to HSM is not supported.

To enable storing of certificates and private keys in HSM, first create a **.cfg** file with a configuration of a PKCS#11 provider. This file has a following format:

- *name=hsm_name*

> *library=path_to_*.so_or_*.dll_library*

> *slot=number_of_slot*

Set library to the PKCS#11 implementation of your HSM and set slot with a number of slots which will be used by ECP component.

In the properties file of ECP component find or create the property *spring.profiles.active* and add profile *ecp-hsm* to the list of profiles.

Then set following properties:

Property	Description	Default value
localKeyStore.implementation.beanName	If ECP component is Component Directory or Endpoint, set value to *localKeystoreHSMImpl*. If ECP component is Broker, set value to *localKeyStoreHSMFiles*	N/A
ecp.hsm.enabled	Create this property only if ECP component is Broker. Then set its value to true	false
ecp.hsm.configPath	Path to the PKCS#11 provider configuration file.	N/A
ecp.hsm.libraryPath	Path to the PKCS#11 implementation (.so or .dll) of your HSM. Value must be the same as in PKCS#11 provider configuration file.	N/A
ecp.hsm.slot	Slot number. Each ECP component must have its own slot. Value must be the same as in PKCS#11 provider configuration file.	N/A
ecp.hsm.pin	HSM user PIN.	N/A
ecp.hsm.keyPassword	Private keys in HSM will be stored with this password.	N/A

For further configuration of HSM in ECP Broker, proceed to chapter *HSM Configuration*, open ***server.xml*** file in the following path:

- *Windows:*
  > *<install_path>\tomcat\conf\server.xml*
- *Centos/RHEL:*
  > *<install_path>/conf/server.xml*

Find connector for port 8443 and set following attributes at these values:

> *certificateKeystoreFile=""*

> *type="PKCS11"*

> *certificateKeystorePassword="pin"*

> *certificateKeyAlias="ecp_module_auth"*

Replace *pin* with value of *ecp.hsm.pin*.

For Component Directory only, set following attributes at these values:

> *truststoreFile="path-to-authKeystore.jks"*

> *truststorePassword="password"*

Replace *path-to-authKeystore.jks and password* with an actual path and a password to the authentication keystore JKS file.

Save *server.xml* and restart ECP component.

To enable HSM in Internal Broker, add and configure these properties in the AMQPS acceptor connection string in ***broker.xml***:

> *ecpHsmPin*

> *ecpHsmTrustStoreLocation*

> *ecpHsmTrustStorePassword*

Copy ***hsm-ssl-context-factory.jar*** from ECP Endpoint */etc/ecp-endpoint/jms/ssl-hsm* to Internal Broker */opt/eccosp-artemis/lib*.

4.11. Cleaning job for expired certificates

On every ECP component, it is possible to enable cleaning job, which will run periodically and removes all expired certificates from database, JKS files or HSM. Therefore, expired certificates will no longer be visible in GUI. Cleaning job can also be configured in a way, that expired certificates will not be removed immediately, but only when number of days after expiration will reach configured value. Following properties are used for the configuration of the cleaning job:

Property	Description	Default value
ecp.certificatesCleaning.enabled	Enable / disable cleaning job for expired certificates.	false
ecp.certificatesCleaning.interval	Defines period how often the cleaning job will be executed. Value is defined as CRON expression.	0 10 9 * * * (every day at 9:10)
ecp.certificatesCleaning.offset	Expired certificate will not be removed until number of days after its expiration will reach this value.	0

4.12. How to configure Keycloak integration with LDAP

The Keycloak supports the User Federation feature, which allows the integration with LDAP or Active Directory server. To enable this feature and configure the User Federation, please refer to the Keycloak documentation.

Please note that the Active Directory will reject all password change requests when a non-SSL (ldap://) connection is used, therefore it is recommended to configure an SSL connection for this integration. For the certificate configuration, please refer to the Keycloak documentation.

See the next chapter on how to configure the LDAP authentication in ECCo SP.

4.13. How to configure LDAP authentication

It is possible to use the LDAP (Lightweight Directory Access Protocol) server for an authentication on the GUI of Component Directory or Endpoint and for Endpoint Web Services. To use LDAP authentication for the ECP Broker Web Console, see the *Web Console configuration* chapter.

ECP requires either the Active Directory server or a server supporting the standard LDAP protocol. A connection to a server can be made by LDAP or LDAPS (LDAP secured with TLS) protocol. LDAPS requires a truststore JKS file with a public certificate of the LDAP server. If LDAP server requires a client certificate authentication, a keystore JKS file with a client private key and certificate signed by the same CA as a certificate of the LDAP server is required.

Active Directory should run on the Windows Server 2022, because previous versions do not support TLS v1.3.

ECP Endpoint requires that users in the LDAP server are assigned to certain groups, which are mapped to the ECP roles. Group names are not case sensitive. It is possible to configure custom group names. By default, LDAP groups with these names will be mapped to the ECP roles:

- *ECP Admins*
  - *Users from this group have “admin” ECP role*
- *ECP Users*
  - *Users from this group have “user” ECP role*

To enable LDAP authentication on Component Directory or Endpoint, add profile **ecp-ldap** to the list of profiles in *spring.profiles.active* in properties file. Make sure that the profile *disable-user-auth* or *ecp-sso* is not in the list. Users are then managed by the LDAP server instead of the **ecp-user.properties** file, which will be ignored.

Some configuration properties are used for both Active Directory and standard LDAP, other properties are different. These properties are common:

Property	Description	Default value
ecp.ldap.url	URL and a port number of the LDAP server. It must start with “ldap://” or “ldaps://”. If it starts with “ldaps://”, a connection will be secured with TLS. Example: ldaps://domain.com:636	N/A
ecp.ldap.truststoreLocation	Path to a truststore JKS file containing a public certificate of the LDAP server. Required when LDAPS is used.	N/A
ecp.ldap.truststorePassword	Password to the JKS truststore. Required when LDAPS is used.	N/A
ecp.ldap.keystoreLocation	Path to a keystore JKS file containing a client private key and certificate. Required when LDAPS is used, and a client certificate authentication is enabled in the LDAP server.	N/A
ecp.ldap.keystorePassword	Password to the JKS keystore. Required when LDAPS is used, and a client certificate authentication is enabled in the LDAP server.	N/A
ecp.ldap.keyPassword	Password to a private key in the JKS keystore. Required when LDAPS is used, and a client certificate authentication is enabled in the LDAP server.	N/A
ecp.ldap.verifyHost	If set to true, a certificate Common Name or certificate Subject Alternative Name must match the LDAP server URL. Applies only when LDAPS is used.	true
ecp.ldap.roleAdmin	Users from a group with this name will be mapped to the “admin” ECP role.	ECP Admins
ecp.ldap.roleUser	Users from a group with this name will be mapped to the “user” ECP role.	ECP Users

Define the following property if the LDAP server is Active Directory:

Property	Description	Default value
ecp.ldap.domain	Active Directory domain, which will be added to a username.	N/A
ecp.ldap.searchFilter	The LDAP filter string to search for the user being authenticated. Occurrences of {0} are replaced with the username@domain. Occurrences of {1} are replaced with the username. When AD uses *sAMAccountName* instead of *userPrincipalName*, use this string: *(&(objectClass=user)(sAMAccountName={1}))*	(&(objectClass=user)(userPrincipalName={0}))

If *ecp.ldap.domain* is not defined, a connection will be made with a standard LDAP authentication instead. Following properties are used then:

Property	Description	Default value
ecp.ldap.base	Base DN (distinguished name), from where the LDAP server searches users and groups. Example: dc=domain,dc=com	N/A
ecp.ldap.username	LDAP administrator username.	N/A
ecp.ldap.password	LDAP administrator password.	N/A
ecp.ldap.pattern	A user search pattern, {0} will be replaced with a login username.	uid={0},ou=users
ecp.ldap.groupBase	LDAP groups search base. Used only by ECP Endpoint.	ou=groups
ecp.ldap.roleSearchMatching	The filter used in the groups search, with the filter argument {0} being the full DN of the user. Used only by ECP Endpoint.	uniqueMember={0}
ecp.ldap.roleSearchSubtree	If set to true, a subtree scope search of groups will be performed, instead of the single-level search. Used only by ECP Endpoint.	false

ecp.ldap.rolePrefix	Sets the prefix which will be prepended to the role values loaded from the LDAP directory. Used only by ECP Endpoint.	Empty string
ecp.ldap.passwordAttribute	Name of a password attribute, which will be used when the password comparison authentication is used.	userPassword
ecp.ldap.useServerAuthentication	When set to true, the bind authentication is used, which means that the authentication is performed by LDAP server. Otherwise, the password comparison authentication is used, where LDAP server sends a stored password of the matched user to the client, where the authentication is performed. This mode should always be used with LDAPS, or for testing purposes only.	true

4.14. How to integrate ECP into existing ELK stack

4.14.1. Prerequisites

- *Existing ELK stack*
- *ECP ELK integration zip file, which includes following files:* ***docker-compose.yml****,* ***filebeat.yml*** *and* ***logstash.conf***

4.14.2. ELK configuration process

1. *To collect ECP logs, Filebeat container must be deployed to a node where ECP component is installed.*
2. ***docker-compose.yml*** *shows example for deployment of filebeat container*
3. *Volumes of filebeat container must be mapped to paths of log file or folder of ECP component which you want to track in ELK. Example:*
   - */var/log/ecp-endpoint/ecp.log:/usr/share/ecp-endpoint/ecp.log*

   Path to ***filebeat.yml*** file must also be mapped.

4. *Open ./filebeat/**filebeat.yml***
5. *In paths section, add all paths to log files inside filebeat container mapped in volumes. Example:*

   paths:

   > */usr/share/ecp-endpoint/ecp.log*

6. *Edit hosts value in output.logstash to match address and port of your Logstash instance.*
7. *logstash.conf shows example of configuration, which you can use to edit your Logstash configuration. Input section must contain beats section with port of Logstash instance exposed to Filebeat instances. Filter section defines matchers for four log patterns, which are used by ECP components.*

4.15. How to log client IP behind proxy

Client IP address is logged in audit log messages. When a reverse proxy is used, IP address of the proxy will be logged instead of a real client IP address. To log a real client IP address behind the proxy, navigate to your installation directory and locate the following file:

- *Windows:*
  > *<install_path>\tomcat\conf\server.xml*
- *Centos/RHEL:*
  > *<install_path>/conf/server.xml*

Find the line <Valve className="org.apache.catalina.valves.RemoteIpValve" internalProxies="" /> and add IP address of the reverse proxy as a value of the *internalProxies* attribute. Value of *internalProxies* is a regular expression, so multiple IP addresses must be separated by “|” character. Example of a configuration with multiple proxies:

<Valve className="org.apache.catalina.valves.RemoteIpValve" internalProxies="192.168.0.10|192.168.0.11|192.168.0.12" />

Save *server.xml* and restart ECP component.

4.16. Compression job for Derby or PostgreSQL database

Default embedded database for ECP component is Apache Derby. In this database system, it may happen that component_directory table use a lot of disk space. ECP provides compression job, which will periodically call Derby system procedure SYSCS_UTIL.SYSCS_COMPRESS_TABLE for component_directory table. This system procedure will reclaim an unused allocated space in the table. If there is no unused allocated space, there will be no change of database disk usage after the run of compression job. All outbox and inbox messaging routes are suspended until the compression job is finished. This is necessary to prevent deadlocks in the component_directory table if the compression and message processing would run at the same time. Once the job is finished (or the job failed in case of an exception), all suspended routes are resumed, and the message processing can continue.

PostgreSQL uses pg_largeobject table for storing large object, which are referenced from other tables. When a row from a parent table is deleted, a large object will remain stored in pg_largeobject table. PostgreSQL provides VACUUM command, which deletes unreferenced object and reclaim disk space. ECP compression job calls this command for pg_largeobject table.

ECP database compression job automatically detects whether Derby or PostgreSQL is used, and it is ignored for other database systems. It is not recommended to run this job very often; default interval is once a day. Use these properties for configuration of the compression job:

Property	Description	Default value
ecp.db.compressionJobEnabled	Enable / disable database compression job.	true
ecp.db.compressionInterval	Defines period how often the compression job will be executed. Value is defined as CRON expression. Default interval is once a day.	0 30 9 * * * (every day at 9:30)

4.17. How to enable auditing of the configuration files

Auditing of the access to the configuration files must be configured at the operating system level. Configuration files are the ***.properties*** and ***.xml*** files located at these directories:

Component Directory, Endpoint:

- *Windows:*
  - *<install_path>*

> *<install_path>\tomcat\conf*

- *Centos/RHEL:*
  > */etc/ecp-directory or /etc/ecp-endpoint*
  > */usr/share/ecp-endpoint/conf or /usr/share/ecp-directory/conf*

Broker:

> *<install_path>*
> *<install_path>\activemq\conf*

In Windows, make sure that auditing is enabled at the server level:

- *Go to Administrative Tools > Local Security Policy*
- *In the left panel, expand Local Policies and then select Audit Policy*
- *In the right panel, right click on Audit object access and select Properties*
- *Enable Success and Failure*

Then enable auditing for each file you wish to audit:

- *Right click on the file, select Properties, select Security tab, click Advanced*
- *Select Auditing tab and add all users all groups you wish to audit*

In Centos/RHEL, *auditd* service can be used for auditing. Open */etc/audit/rules.d/**audit.rules*** file and add a rule for each file you wish to audit, for example:

-w /etc/ecp-endpoint/ecp.properties -p war -k ecp

Then restart the *auditd* service:

- *systemctl restart auditd*

4.18. How to configure a maximum request size

Except the file upload requests, a size of the ECP GUI request cannot exceed 2 megabytes by default, otherwise the request will be rejected.

Use this property to configure the maximum request size:

Property	Description	Default value
ecp.maxRequestSize	Maximum ECP GUI request size in bytes.	2097152

4.19. How to enable full stack trace logging

A stack trace of ECP exceptions is logged in a limited form by default. To enable the full stack trace logging, add the following property in this location:

> *Linux system: /etc/systemd/system/<ecp-directory.service.d or ecp-endpoint.service.d>/**env.conf** or **setenv.sh**, if ECP is not running as a service*

> *Windows system: Java Options in Java tab (command <install_path>/tomcat/bin/tomcat10w.exe //ES/ecp-endpoint)*

Property	Description	Default value
ecp.logFullStackTrace	When set to true, all stack trace steps will be logged, each on the separate line. When set to false, stack trace of ECP exceptions will be logged in one line, and less important stack trace steps will be omitted.	False

4.20. Database backup and restore

ECP Component Directory and ECP Endpoint allows to create a database backup, which could be later used to restore the ECP component back to the state where a backup was created, or to import it to a new installation of the ECP component. In ECP Endpoint, the backup does not contain the messages.

The backup process creates a CSV file for each backed-up database table, and all CSV files are put into one zip file. Its filename contains the component code and a current timestamp.

The restore process validates that a provided zip file contains all CSV files and then replaces data in each backed-up database table. The process is atomic, so in case of an error, no data are replaced in any table. After a successful restore, the component reloads its context.

ECP Component Directory creates a backup for these database tables:

- *application_property*
- *component_directory*
- *component_statistics*
- *issued_certificates (included in registration_requests CSV)*
- *local_key_store*

- *message_path*
- *message_path_receiver (included in message_path CSV)*
- *message_path_sender (included in message_path CSV)*
- *message_type*
- *pending_edit_directories*
- *pending_removal_directories*
- *registration_requests*
- *synchronized_directories*

ECP Endpoint creates a backup for these database tables:

- *application_property*
- *component_directory*
- *local_key_store*
- *message_path*
- *message_path_sender (included in message_path CSV)*
- *message_type*
- *message_upload_route*
- *messaging_statistics*
- *registration_store*

The backup zip file can be either created from the GUI in the Settings page, or by sending a POST request to *<ECP component url>/settings/database/backup* to the ECP component API.

The restore can be performed by uploading a backup zip file using the GUI in the Setting page or in the first page of the Registration process. It can be also uploaded by sending it with a PUT request to *<ECP component url>/settings/database/restore* to the ECP component API.

In case of ECP Component Directory, connected Endpoints in version 4.15.0 or lower should manually push their configuration to the Component Directory, in case there was a change in their message paths or contact information since the Component Directory backup was created.

4.21. How to log to the console

To enable the console as a logging output for ECP Endpoint or ECP Component Directory, find the property *spring.profiles.active* in the ***ecp/ecp-directory.properties*** file and add the *console-logging* profile to the list of profiles.

With enabled logging to the console, ECP application keeps logging to the log files as before.

The logging to the console is useful when ECP is deployed in the Docker or Kubernetes container as it will log the application logs to the container/pod logs.

5. ECP Endpoint Administration

5.1. User authentication

To prevent unauthorized access, ECP endpoint can be configured to require users to log in and to include username and password in the header of the Web Service request. It is strongly recommended to use HTTPS connection when user authentication is enabled. See chapter *Enable HTTPS* for details how to enable HTTPS.

A list of allowed users is defined in ***ecp-users.properties*** file in the configuration directory. There are two user roles: admin and user. Only admin role has access to the Settings page and all its functionality and can use web services related to message paths.

A new installation comes with one predefined user: admin/password. It is strongly recommended to replace this user with your own or at least change the password to a more secure one.

The pattern is:

- *ecp.endpoint.users[index].login=user*
- *ecp.endpoint.users[index].password=password*
- *ecp.endpoint.users[index].role=role*

The index must be a whole number from a sequence starting with zero. Example:

- *ecp.endpoint.users[0].login=janedoe*
- *ecp.endpoint.users[0].password=securepassword*
- *ecp.endpoint.users[0].role=admin*
- *ecp.endpoint.users[1].login=johndoe*
- *ecp.endpoint.users[1].password=secret*
- *ecp.endpoint.users[1].role=user*

5.1.1. GUI Authentication

GUI authentication is enabled by default. To disable the user authentication in Endpoint GUI, find the property *spring.profiles.active* in the ***ecp.properties*** file and add the *disable-user-auth* profile to the list of profiles.

5.1.2. Web Services Authentication

Web Services authentication is enabled by default. To disable the user authentication in Web Services, find the property *spring.profiles.active* in the ***ecp.properties*** file and add the profile *disable-ws-auth* to the list of profiles.

With enabled Web Service authentication, every web service request must include the header following the WS-Security protocol. Example:

```
<soapenv:Header>
<wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd" wsse:mustUnderstand="1">
        <wsse:UsernameToken>
            <wsse:Username>admin</wsse:Username>
            <wsse:Password>password</wsse:Password>
        </wsse:UsernameToken>
    </wsse:Security>
</soapenv:Header>
```

Matching username and password must be found in ***users.properties*** file (by default), or in LDAP server (when *ecp-ldap* profile is used).

5.1.3. Monitoring Authentication

By default, an access to all actuator endpoints exposed on */ECP_MODULE/actuator/** URL is protected by the user authentication. It is possible to disable the authentication for individual endpoints by adding the endpoint name to the comma-separated list in *ecp.disabledAuthenticationActuatorEndpoints* property in the ***ecp.properties*** file.

The *management.endpoints.web.exposure.include* property contains the list of names of the exposed actuator endpoints.

5.1.4. Passwords in Hash

ECP Endpoint supports password stored in SHA512 encryption with salt prefix.

Example:

ecp.endpoint.users[index].password=6<salt_prefix>$<hashed_password>

5.1.4.1. How to generate hashed password with salt

Prerequisites: installed command line tool ‘openssl’ which is available for all OS.

The hashed password is generated with command:

openssl passwd -6 -salt <salt>

- *option -6 indicates algorithm SHA512*
- *option -salt indicates selected salt prefix*

Result of generating hash in this template:

- *6<salt_prefix>$<hashed_password>*

For example:

 # openssl passwd -6 -salt test

Password: <enter password>

6test$zW0OdEUeeqYAEUmnXZ3CcdTnF867qiC1306IvgaUfV7MqmQ0lBIaxFBkpxpaEqO5eLJrDe2F8PaZXYM0eTP04.

5.2. How to configure unsuccessful login attempts

To ensure better security, Endpoint supports a configurable limit of the number of unsuccessful login attempts. After the limit is reached, login will be locked for a period of time.

Property	Description	Default value
ecp.loginLock.attempts	Limit of unsuccessful login attempts	3
ecp.loginLock.lockTime	Lock time in seconds	900

5.3. How to configure message paths

After installation of ECP endpoint there will be no message paths present, and it will not be possible to send or receive messages to or from the current Endpoint. The following instructions describe how to add a new message path.

5.3.1. How to add a new message path

Open the Endpoint web interface and navigate to page settings. Locate the paths section and click the new path button. A new path dialog box will open.

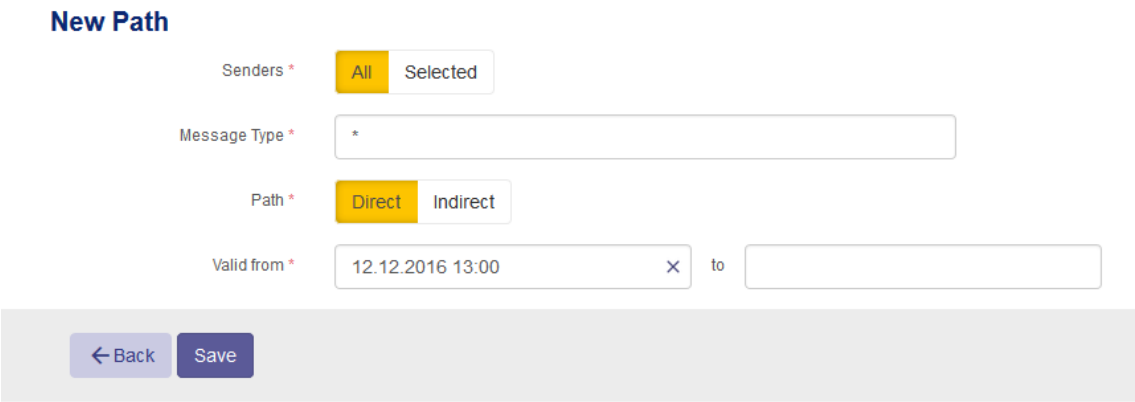

Figure 1: New Path

Indirect message paths can be created for existing intermediate components (typically ECP broker), which must allow communication with endpoint. See also configuration of broker filter: How to configure component filter.

Direct message paths can be created only when direct messaging is enabled on endpoint. For instructions on how to enable direct messaging on endpoint see How to enable direct communication.

Message path message type support's the definition of message type with a wild card character. For message path matching rules description see also chapter *Message type matching rules*.

When a new message path is successfully created, it will be distributed to other ECP components through the component directory synchronization. Please note that this process is asynchronous and requires time to complete.

When you want to remove a message path, execute these two steps:

1. *Instead of deleting the message path, change its validity to the near future so that all components in the network will know that a given message path will no longer be valid and won't allow any message to be sent using this path.*
2. *Create a new message path and set its validity start to the date and time when the removed message path validity ends.*

5.3.2. Import message paths in a file

Endpoint support batch import of message paths defined in a JSON file. To import message paths file, click on the "Import Paths" button in Settings page of Endpoint GUI. File structure reflect message path structure, single record represents one message path.

Attribute	Type	Description	Required
messageType	string	A text, possibly with a wildcard (*) at the end (e.g. ABC*), which indicates the message-type of the messages that can use the path.	True
path	string	Format: {DIRECT | INDIRECT }:<broker code>, <broker code> is mandatory when an INDIRECT path.	True
senderComponent	string[]	The collection of the codes of the sender endpoints that can use the path to send a message. A wildcard (*) means any endpoint.	True
validFrom	dateTime	The date time (inclusive) from which the path is usable.	True
validUntil	dateTime	The date time (exclusive) until the path is usable, forever when not set.	False

Example of a message paths file. Times of *validFrom* and *validUntil* must be defined in the time zone, which is set in GUI:

```
[{
  "messageType": "TYPE1",
  "path": "DIRECT",
  "senderComponent": ["*"],
  "validFrom": "2019-05-20T12:00:00",
  "validUntil": "2020-05-20T12:00:00"
},
{
  "messageType": "TYPE2",
  "path": "INDIRECT:B1",
  "senderComponent": ["EDC1","EDC2"],
  "validFrom": "2019-05-20T12:00:00"
}]
```

Import procedure starts with message paths validation. If single validation error occurs, import procedure is interrupted and validation result is returned to the import caller. Successfully imported message paths are pushed by the Endpoint to the Component Directory.

5.3.3. How to configure processing of centrally managed message paths

Following properties configures processing of centrally managed message paths. Synchronization job is paired together with synchronization job for message types (see chapter TTL per Message Type), so configuration of following properties changes configuration of both jobs:

Property	Description	Default value
ecp.endpoint.messagePath.synchronization.enabled	Enable / disable synchronization of centrally managed message paths and message types with Component Directory	true
ecp.directory.client.synchronization.messagePathSynchronizationInterval	Defines period how often the synchronization routine will be run. Value is defined as CRON expression.	0 */5 * * * * (every fifth minute)

5.3.4. Message path selection

When there are multiple message paths matching a message type of a message, the most specific message path is selected. Examples of the message path selection for a message type TEST:

- *Equal message type (with no wildcard) is the most specific.*
  - *TEST*
- *Message type plus wildcard is the second most specific.*
  - *TEST**
- *Message type substring plus wildcard is the third most specific. If there are multiple message paths with a message type substring, a message path with a longest substring is selected.*
  - *TES* – it will be selected*
  - *TE* – it will not be selected*
- *Wildcard only is the least specific.*
  - *

5.4. How to configure FSSF

FSSF can be used for both sending and receiving of messages. It is also possible to configure endpoint FSSF only for message receiving/sending. Note that all directories used by FSSF should exist and have appropriate permissions for file read/write operations (the user under which ECP is running must have read/write permission). It is also strongly recommended that directories used by FSSF are located outside of the installation folder.

If you are using non UTF-8 characters in filenames, make sure that host system locale settings match the used characters, to prevent issues with wrong filenames encoding.

To enable FSSF set the configuration property "*ecp.endpoint.fssf.enabled*" value to true.

Property	Description	Default value
ecp.endpoint.fssf.enabled	Enable / disable FSSF interface on endpoint.	false

5.4.1. Sending messages through FSSF

To specify the outgoing directory for sending messages through FSSF set the configuration property "*ecp.endpoint.fssf.outgoingDirectory*" value to the desired directory path (e.g. /opt/fssf/out).

Together with the outgoing directory, the directories for message processing log and errors must also be specified.

- *To specify the message processing log directory set the configuration property "ecp.endpoint.fssf.logDirectory" value to the desired directory path.*
- *Single log file per day for message processing can be enabled by setting the property "ecp.endpoint.fssf.useSingleLogPerDay" value to true. The default FSSF logging setting is one log file per sent message.*
- *To specify the FSSF error directory where invalid files are sent, set the property "ecp.endpoint.fssf.outgoingErrorDirectory" value to the desired directory path.*

All properties required for sending messages over FSSF are listed in the table below:

Property	Description	Default value
ecp.endpoint.fssf.outgoingDirectory	Outgoing directory path.	N/A
ecp.endpoint.fssf.logDirectory	Message processing logs directory path.	N/A
ecp.endpoint.fssf.outgoingErrorDirectory	Unprocessed messages directory path.	N/A

ecp.endpoint.fssf.useSingleLogPerDay	Enable single log file per day for all processed messages.	false

5.4.2. Receiving messages through FSSF

The following receive strategies can be configured:

- *Incoming messages are received to a single directory regardless of their type.*
- *Incoming messages are routed to different directories based on their message type or group of messages types.*

5.4.2.1. Single incoming directory

To specify a single incoming directory for receiving messages regardless of their type, set the configuration property "*ecp.endpoint.fssf.incomingDirectory[0].incomingDirectory*" value to the desired directory path and set the message type wild character * as a value of the property *ecp.endpoint.fssf.incomingDirectory[0].messageTypes.*

5.4.2.2. Multiple incoming directories

To specify incoming directory per message type(s), the following tuples of properties are required:

> *ecp.endpoint.fssf.incomingDirectory[n].incomingDirectory = <directory>*

> *ecp.endpoint.fssf.incomingDirectory[n].messageTypes = <message type>*

Where *n* represents a number from a continuous series of integers starting from 0.

Multiple message types must be separated by a comma. For message type matching rules description see Message type matching rules.

Property	Description	Default value
ecp.endpoint.fssf.incomingDirectory[n].incomingDirectory	Incoming directory path for messages specified by message types defined by next property.	N/A
ecp.endpoint.fssf.incomingDirectory[n].messageTypes	Incoming directory message type(s). Multiple types must be separated by a comma. Message type may contain a wildcard character (*). See also section *Message type matching rules*.	N/A
ecp.endpoint.fssf.incomingDirectory[n].extension	Default file extension will be added to a filename if a received file has no file extension for messages specified by message types defined by previous	N/A

	property.

5.4.2.3. Default file extension

Default file extension can be configured, which will be added to a filename during receive in case that a received file has no file extension and no matching extension was found in *ecp.endpoint.fssf.incomingDirectory[n].extension* property. Use this property to configure the default file extension:

Property	Description	Default value
ecp.endpoint.fssf.defaultExtension	Default file extension will be added to a filename if a received file has no file extension.	N/A

5.4.3. Message type matching rules

Message type configuration properties support the matching of message type strings against a wildcard character * or a combination of wildcard character and string.

Possible message type specifications are:

- *wildcard character **
- *message type plus wildcard*
- *message type substring plus wildcard.*
- *plain message type*

The order of properties list specified by configuration is important. The first property matched is selected e.g.

> *Directory[0].messageType = type**

> *Direcotry[1].messageType = typeA*

> *Directory[2].messageType = **

Given message type: typeA first property is chosen by system as matching.

Given message type: A the last property is chosen by system as matching.

Obviously, when multiple message types separated by a comma are configured within a single property, message types order within the single property is not important. See the following example:

> *Directory[0].messageType = typeA, typeB, typeC*

However, it is illogical to configure:

> *Directory[0].messageType = type, **

Because the wildcard character matches any message type.

5.5. How to configure AMQP API

To enable the AMQP interface set the property "*ecp.endpoint.amqpApiEnabled*" value to true. Enabled AMQP interface with default settings provides three AMQP queues available for external systems:

- *ecp.endpoint.outbox - queue for sending messages*
- *ecp.endpoint.inbox – queue for receiving messages*
- *ecp.endpoint.outbox.reply – queue for reply messages*

The default inbox queue receives all messages regardless of type.

Property	Description	Default value
ecp.endpoint.amqpApiEnabled	Enable AMQP API by setting this property value to true	false

5.5.1. Setup of multiple receive queues

Several receive queues can be configured for message types or groups of types. The received message is then routed by message type to a queue with a matching message type following the rules described in section *Message type matching rules*.

To specify the receive queue per message type(s), the following tuples of properties are required to be set: (one tuple per queue):

- *ecp.endpoint.routes.incomingQueue[n].incomingQueue = <queue name>*
- *ecp.endpoint.routes.incomingQueue[n].messageTypes = <message type>*

Where *n* represent a number from a continuous series of integers starting from 0.

Multiple message types must be separated by a comma. For message type matching rules description see *Message type matching rules*.

Property	Description
ecp.endpoint.routes.incoming Queue[n].incomingQueue	Incoming queue for messages specified by message types defined by next property.
ecp.endpoint.routes.incoming Queue[n].messageTypes	Incoming queue message type(s). Multiple types must be separated by a comma. See also section *Message type matching rules*.

5.5.2. How to enable AMQP send handler

ECP Endpoint comes with default implementation of send handler for AMQP API. The AMQP send handler is pushing incoming SendEvents into queue named:

> *ecp.endpoint.send.event*

To enable the AMQP send handler set handler properties as described at table below:

Property	Description
ecp.endpoint.sendHandler [n].beanName	Specify AMQP send handler bean name. This value must be set to: *amqpApiSendHandler* to enable the AMQP send handler.
ecp.endpoint.sendHandler [n].typeName	This property specifies send handler message types. For message type matching rules description see section *Message type matching rules*. Use the wildcard character * to enable handler invocation for all messages regardless of their message types.

Where *n* represents a number from a continuous series of integers starting from 0.

5.5.3. How to secure AMQP API

By default, the ECP Endpoint AMQP API authentication and transport level security is enabled by having the *internalBroker.useAuthentication* property set to true in ***ecp.properties***.

Following parameters are automatically added to the connection URI. You can override default value by adding the parameter to the connection URI in *internalBroker.urls* or *ecp.urls* property.

Property	Description	Default value
transport.trustStoreLocation	Path to truststore file.	Value of ecp.authKeystore.location
transport.trustStorePassword	Password of truststore file.	Value of ecp.authKeystore.password
transport.keyStoreLocation	Path to keystore file.	Value of ecp.authKeystore.location
transport.keyStorePassword	Password of keystore file.	Value of ecp.authKeystore.password
transport.keyAlias	Alias of the key in keystore file.	Value of ecp.authKeystore.authAlias

transport.verifyHost	Hostname verification is disabled by default because ECP authentication keystore do not include a server name.	false

All configuration properties in ***ecp.properties*** for the authentication and Internal Broker SSL connection:

Property	Description	Default value
internalBroker.useAuthentication	Turns on login-based authentication and internal broker SSL.	false
internalBroker.auth.user	Sets endpoint username for Internal Broker.	N/A
internalBroker.auth.password	Endpoint user password for Internal Broker.	N/A
internalBroker.keystore.location	Path to the KeyStore with SSL certificate. ***authKeystore.jks*** file path can be used.	N/A
internalBroker.keystore.password	SSL keystore password.	N/A
internalBroker.keystore.authAlias	SSL certificate alias. When ***authKeystore.jks*** is used, alias *ecp_module_auth* can be used.	N/A

5.5.4. How to disable AMQPS

To disable login-based authentication and transport level security (AMQPS), the property *internalBroker.useAuthentication* has to be set to false in ***ecp.properties***.

If a failover is configured in *internalBroker.urls*, add the *failover.amqpOpenServerListAction* parameter and set it to *IGNORE*. Example of *internalBroker.urls*:

> *failover:(amqp://host1:5672,amqp://host2:5672)?failover.amqpOpenServerListAction=IGNORE*

When disabled, AMQPS in the Internal Broker acceptor configuration must also be disabled. See the *How to disable AMQPS and authentication* subchapter in the *Internal Broker configuration* chapter for more details.

5.6. How to configure archiving

ECP Endpoint provides file system archiving capabilities, however an additional custom archiving plugin can be implemented and used instead of the default file system archiving component.

5.6.1. Setup of file system archiving

To enable the file system archive handler component set the property "*ecp.endpoint.archiveEnabled*" value to true. The file system archiving component saves message content and message metadata to separate files created within configured directories.

Additional file system handler properties must be set to enable archiving functionality. See table below:

Property	Description	Default value
ecp.endpoint.archive Enabled	To enable archiving set value of this property to true.	false
ecp.endpoint.archive Handler.beanName	Use this property to specify selected archive handler. To enable file system archive handler set value to *fileSystemArchiveHandler*.	N/A
ecp.endpoint.archive Handler.typeName	This property specifies message types to be archived. For message type matching rules description see section *Message type matching rules*. Use the wildcard character * to enable archiving of all messages regardless of their message types.	N/A
ecp.endpoint.archive. binaryPathExpression	Define directory path for message content archive files. Path can be constructed dynamically from message properties. See available path expression described in section *List of available path expressions*.	N/A
ecp.endpoint.archive. metadataPathExpres sion	Define directory path for message metadata archive files. See available path expression described in section *List of available path expressions*	N/A

5.6.1.1. List of available path expressions

The following table shows available message properties expressions that can be used for file system archiving paths construction:

Expression	Description
#{#sender}	Message sender code.
#{#receiver}	Message receiver code.
#{#messageType}	Message type.
#{#messageID}	Message ID.
#{#extension}	Message extension (file extension for messages sent over FSSF).
#{#timeSent.Year}	Year part of message sent date.
#{#timeSent.MonthValue}	Mont part of message sent date.
#{#timeSent.DayOfMonth}	Day part of message sent date.

The recommended path expression pattern for content archiving is:

> */#{#timeSent.Year}/#{#timeSent.MonthValue}/#{#timeSent.DayOfMonth}/#{#sender}_#{#receiver}_#{#messageType}_#{#messageID}.#{#extension}*

The recommended path expression pattern for metadata archiving is:

> */#{#timeSent.Year}/#{#timeSent.MonthValue}/#{#timeSent.DayOfMonth}/#{#sender}_#{#receiver}_#{#messageType}_#{#messageID}-metadata.xml*

Note that paths patterns above are not complete and parent archive directory should be added before the beginning of the patterns.

5.6.2. Registration of custom archive handler

Custom implementations of the archiving handler must be configured properly to be used by ECP Endpoint. The implementation class of archive handler can be registered by class name (e.g. *eu.custom.archive.handler.HandlerClassName*).

For how to deploy custom archive plugin instructions see section *How to add plugins to ECP*.

Property	Description	Default value
ecp.endpoint.archiveHandler.className	Specify custom handler implementation by Java class name (e.g. eu.custom.archive.handler.HandlerClassName)	N/A

ecp.endpoint.archiveHandler.typeName	This property specifies message types to be archived. For message type matching rules description see section *Message type matching rules*. Use the wildcard character * to enable archiving of all messages regardless of their message types.	N/A

5.7. How to configure receive handlers

Multiple handlers can be registered for different message types. The receive handler is invoked when a standard message with matching message type is received by ECP Endpoint. Handler implementations are configured as listed in the following properties:

Property	Description	Default value
ecp.endpoint.receiveHandler[n].className	Specify custom handler implementation by Java class name (e.g. eu.custom.receive.handler.HandlerClassName)	N/A
ecp.endpoint.receiveHandler[n].typeName	This property specifies receive handler message types. For message type matching rules description see section *Message type matching rules*. Use the wildcard character * to enable handler invocation for all messages regardless of their message types.	N/A

Where *n* represents a number from a continuous series of integers starting from 0.

For how to deploy receive handler instructions see section *How to add plugins to ECP*.

5.8. How to configure send handlers

Multiple handlers can be registered for different message types. The send handler is invoked when an acknowledgement message with matching message type is received by ECP Endpoint. Handler implementations are configured as listed in the following properties:

Property	Description	Default value
ecp.endpoint.sendHandler [n].className	Specify custom handler implementation by Java class name (e.g. eu.custom.send.handler.HandlerClassName)	N/A
ecp.endpoint.sendHandler [n].typeName	This property specifies send handler message types. For message type matching rules description see section *Message type matching rules*. Use the wildcard character * to enable handler invocation for all messages regardless of their message types.	N/A

Where *n* represents a number from a continuous series of integers starting from 0.

For how to deploy send handler instructions see section *How to add plugins to ECP*.

5.9. How to configure message expiration

Messages expire after a defined time to live (TTL) value. TTL value is defined in seconds. If the message expires while it is in the Internal Broker or in ECP Broker, it will be moved to the *DLQ* queue.

Property	Description	Default value
ecp.endpoint.messageTtl	This property specifies TTL in seconds.	14x24x60x60 (14 days)

Expired messages stored in the database of ECP Endpoint are processed by the message expiration job. Its purpose is to change the state of expired messages to final state – FAILED. The interval for executing this job can be set by the property described below:

Property	Description	Default value
ecp.messagebox.messageExpirationRunPeriod	Define period how often the message expiration job [ms] will be run.	60x60x1000 (1 hour)

5.9.1. TTL per Message Type

It is possible to set different TTL per Message Type. In Settings page there is section Message Types with the table of specified message types and their TTL. Message types which are not listed in the table have TTL from a value of *ecp.endpoint.messageTtl* property.

By clicking on "+ New Message Type" button, a modal window will open with fields for Message Type and Time to live. Time to live value must contain at least one number with time unit for days, hours, minutes, or seconds. Examples of Time to live values: 2d 12h 30m 15s, 5h 20m, 7d

New Message Type must not already exist in the table. Existing message types can be edited or deleted from the table.

5.9.1.1. Synchronization of Message Types

TTL per Message types can be also configured on Component Directory. Endpoints periodically run synchronization job to obtain Message types from their home Component Directory. If any received message type already exists on Endpoint, local message type takes precedence over message type from Component Directory.

Synchronization job is paired together with synchronization job for centrally managed message paths (see chapter *How to configure message paths*), so configuration of following properties changes configuration of both jobs:

Property	Description	Default value
ecp.endpoint.messagePath.synchronization.enabled	Enable / disable synchronization of centrally managed message paths and message types with Component Directory.	true
ecp.directory.client.synchronization.messagePathSynchronizationInterval	Defines period how often the synchronization routine will be run. Value is defined as CRON expression.	0 */5 * * * * (every fifth minute)

5.10. How to configure message priority

Priority is assigned to messages based on message type. Supported priority values are from 0 (lowest) to 9 (highest). Message priorities are configured as listed in the following properties:

Property	Description	Default value
ecp.endpoint.priorityConfiguration[0].priority	Message priority in range 0-9 inclusive is supported.	N/A

ecp.endpoint.priorityConfiguration[0].messageType	This property specifies which priority has message type. For message type matching rules description see section *Message type matching rules*.	N/A

Where *n* represents a number from a continuous series of integers starting from 0.

5.11. How to configure logging

You can change the log file location or configure the logger severity for a particular package through the system configuration.

5.11.1. Configure log file location

ECP application log file location can be set via the configuration property *logging.file.path* and its name can be set via the configuration property *logging.file.name*.

Example:

- *logging.file.path=/var/log*
- *logging.file.name=/var/log/ecp.log*

5.11.2. Configure logger severity

Logger severity for a package can be set via the configuration property in the system configuration file. The pattern is *logging.level.<package>=SEVERITY*. Severity can be one of the following values: TRACE, DEBUG, INFO, WARN, ERROR. Please bear in mind that setting logger severity to TRACE or DEBUG will produce a huge quantity of log messages resulting in increased storage usage and may also slow down the application. Setting logger severity to TRACE or DEBUG should be for diagnostic purposes only.

Example:

- *logging.level.eu.entsoe.ecp.endpoint.fssf=DEBUG*

5.11.3. Configure log size and rotation

Logs are automatically compressed after reaching size of 50MB. Maximal size of logs is set to 70MB (compressed files and the live one). This behaviour can be changed by configuration.

Following configuration parameters were added to the ECP component. These parameters are present in the **ecp-logback.xml**.

Log files are stored with following naming convention: ecp.log and archived and saved with following naming convention:

> *ecp.log.yyyy-mm-dd.nn.gz, where value "nn" is the number of created archive.*

Properties already available in the ecp-logback.xml configuration file:

> *# files with more than 50MB will be archived, default 50MB*

> *<property name="LOGBACK_ROLLINGPOLICY_MAX_FILE_SIZE" value="50MB"/>*

> *# maximum number of days the archives will be stored, default 7*

> *<property name="LOGBACK_ROLLINGPOLICY_MAX_HISTORY" value="7"/>*

> *# maximum size of all archived files in the log directory, default 70MB*

> *<property name="LOGBACK_ROLLINGPOLICY_TOTAL_SIZE_CAP" value="70MB"/>*

More available properties:

> *# Whether to clean the archive log files on startup, if true, all log files will be deleted during start of ECP, default value false*

> *<property name="LOGBACK_ROLLINGPOLICY_CLEAN_HISTORY_ON_START" value="true"/>*

> *# Pattern for rolled-over log file names, files will be automatically compressed to the Gunzip archive (.gz), in case it does to ends with .gz the log is not compressed.*

> *<property name="LOGBACK_ROLLINGPOLICY_FILE_NAME_PATTERN" value="${LOG_FILE}.%d{yyyy-MM-dd}.%i.gz"/>*

5.11.4. Log full package names

Class package names are abbreviated in logs by default because of 40 characters limit. This limit can be changed in the **ecp-logback.xml**.

Find a line with this property:

> *<property name="FILE_LOG_PATTERN" value=…*

Inside the *value* string find this sequence:

> *40logger{39}*

Replace 40 and 39 with higher numbers. The first number is a limit of how many characters are displayed in the log message, the second number is a limit used by a logger to abbreviate package names. The class name is never abbreviated, even if its length is longer than the limit. Package names may be shortened to at most a single character. When the abbreviated packages and class string is still longer than the first limit, then only the rightmost characters will be printed.

5.12. Configure HTTPS

It is possible to setup Transport Layer Security (TLS) to secure Endpoint GUI and business application access. HTTPS with TLSv1.3 protocol is enabled by default.

5.12.1. Tomcat configuration

To configure HTTPS connection, navigate to your installation directory and locate the following files:

- *Windows:*
  - *<install_path>\tomcat\conf\server.xml*
- *Centos/RHEL:*
  - *<install_path>/conf/server.xml*

HTTPS is enabled when there is a HTTPS connector definition in Tomcat's ***server.xml***. The parameter *certificateKeystoreFile* targets to a standard Java keystore file. This file must contain either a public or private key. There is a default keystore present in ECP installation folder, which is replaced by ECP keystore during the registration process. Keypair in the default keystore is valid until 21.02.2026. An example of a default configuration:

```
<Connector
        port="8443"
        protocol="eu.entsoe.ecp.connector.protocol.EcpHttp11NioProtocol"
        sslImplementationName="org.apache.tomcat.util.net.jsse.
        JSSEImplementation"
        maxThreads="150" SSLEnabled="true" maxParameterCount="1000" >
    <SSLHostConfig
        protocols="TLSv1.3"
        ciphers="TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SH
        A256,TLS_AES_128_GCM_SHA256" >
```

```
        <Certificate certificateKeystoreFile="/var/lib/ecp-endpoint/authKeystore.jks"
        certificateKeystorePassword="password" type="RSA" />
    </SSLHostConfig>
</Connector>
```

Values in *protocols* parameter are represented as comma separated list of SSL protocols to support for HTTPS connections. If specified, only the protocols that are listed and supported by the SSL implementation will be enabled. If not specified, the JVM default (excluding SSLv2 and SSLv3 if the JVM enables either or both by default) is used.

Parameter *ciphers* specifies which ciphers can be used for HTTPS connection. If not specified, the JVM default ciphers from protocols specified in *protocols* are used.

A non-secured connector (port 8080) is disabled by default. You can enable it by uncommenting HTTP connector configuration in *server.xml* if you wish to use non-secured communication as well.

5.13. How to enable SOCKS Proxy

Endpoint can be configured to communicate with other ECP components through the SOCKS5 proxy server. The following system configuration properties (see System configuration) are used to setup ECP Endpoint communication through proxy.

Property	Description	Default value
ecp.security.proxy.enabled	Turns the use of proxy on / off.	False
ecp.security.proxy.host	The hostname or IP address of the proxy server.	N/A
ecp.security.proxy.port	The port of the proxy server	1080
ecp.security.proxy.nonProxyHosts	List of hostnames / IP addresses for which the proxy is not used. This can be used, for example, to exclude the communication with Internal Broker, the current server (0.0.0.0,127.0.0.1), or all nodes in ECP failover cluster.	N/A
ecp.security.proxy.username	Optional username for authentication on the proxy server.	N/A
ecp.security.proxy.password	Optional password for authentication on the proxy server.	N/A

ECP was tested with a Dante proxy server (https://www.inet.no/dante/) with the default configuration.

5.14. How to add plugins to ECP

ECP Endpoint plugins (e.g. send, receive or archive handlers) which implement ECP endpoint Java public API can be deployed to ECP Endpoint as Java jar packages. To deploy the plugin on Windows, copy the plugin jar file to *<endpoint install path>/tomcat/webapps/ECP_MODULE/WEB-INF/lib/*.

On Linux, if the ECP Endpoint was just installed or upgraded and was not running yet, it is necessary to unzip *ECP_MODULE.war* located in *<endpoint install path>/webapps/*. Then copy the plugin jar file to *<endpoint install path>/webapps/ECP_MODULE/WEB-INF/lib/*.

In case of ECP Endpoint installed in HA configuration, jar packages must be present at all cluster nodes.

5.15. How to enable direct communication

Before using the direct messaging feature of ECP Endpoint, it must be configured in the connected Internal Broker. Please refer to the *Internal Broker configuration – How to enable direct communication* chapter for the required configuration in the Internal Broker.

The following properties must be set in the ECP Endpoint properties file:

Property	Description	Default value
ecp.endpoint.directMessagingEnabled	To enable direct messaging, set the value of this property to true.	false
ecp.urls	This property specifies the URL of the Internal Broker AMQPS acceptor for the direct messaging. The typical URL pattern looks like: *amqps://<host name>:<port>* Multiple URL's separated by a comma can be configured.	N/A

5.16. How to configure message deleting

There properties can be used for a configuration of the message deleting from the database:

Property	Description	Default value
ecp.messagebox .retentionPeriod	Defines the retention period in milliseconds of messages stored in the database. When the retention period of the archived messages expires, they will be deleted from the database.	1000x60x 60x24x14 (14 days)
ecp.messagebox .messageDeletin gRunPeriod	Define period of how often (in milliseconds) the message deleting routine will be run.	1000*60*6 0 (1 hour)

5.17. How to configure Component Directory synchronization

Synchronization of ECP Endpoint with Component Directory is done by a periodically executed routine that must be properly configured with the following properties:

Property	Description	Default value
ecp.directory.client.synchronization. homeComponentDirectoryPrimaryU rl	Defines URL of primary component directory. This URL will be used: - *for the very first synchronization of the component directory.* - *when the URL of primary CD differs from the URL specified at component directory content which is not reachable.*	N/A
ecp.directory.client.synchronization. directorySynchronizationInterval	Defines period how often the synchronization routine will be run. Value is defined as CRON expression. Before the synchronization, it is checked whether the database compression is running. If yes, synchronization	0 */5 * * * * (every fifth minute)

	will wait for 15 seconds before trying again. If the compression is still running, the synchronization will fail.	
ecp.directory.client.statistics.directorySynchronizationInterval	Defines period how often the routine synchronizing endpoint messaging statistics with Component Directory will be run. Value is defined as CRON expression.	0 */10 * * * * (every tenth minute)

5.18. How to configure message signing

ECP Endpoint can be configured to sign outgoing messages and verify signatures of incoming messages (and is configured to do so by default).

Property	Description	Default value
ecp.security.messageSigner.default.enabled	This property enables / disables message signing. and message signature verification for all messages processed by endpoint regardless their message types.	true

5.18.1. How to configure multiple message signing

Messages can be provided with an additional signature (to default one) created from custom certificates stored at filesystem key store. Configuration of additional (also called special) message signer components is described below:

Property	Description	Default value
ecp.security.messageSigner.special.enabled	This property enables / disables special message signing and message signature verification for all messages processed by Endpoint regardless their message types.	false
ecp.security.keyStore.keyAlias	Special message signer key store alias (e.g. ecp_module_sign).	N/A
ecp.security.keyStore.keyStorePass	Special message signer key store password.	N/A

ecp.security.keyStore.keystorePath	Special message signer key stores directory location. The configured directory will be searched for key store files with the following name pattern: - *SenderKeystore-<endpoint_code>.jks* - *ReceiverKeystore-<endpoint_code>.jks* Sender key stores are used for signature verification in incoming messages. Receiver key stores are used for message signing of outgoing messages.	N/A

5.19. How to configure NAT

Before using network address translation (NAT), at least one record of the NAT table must be defined. Each network address is compared to the NAT table to search a matching record with the same network code, IP address and port. If a match is found, the IP address and the port is translated.

Property	Description	Default value
ecp.natEnabled	This property enables (true) or disables (false) NAT.	false
ecp.natTable[i].network	Network code of the network address. If not provided, default code is used.	N/A
ecp.natTable[i].sourceIp	Source IP address to match. This property is mandatory.	N/A
ecp.natTable[i].destinationIp	Source IP will be translated to this IP address. This property is mandatory.	N/A
ecp.natTable[i].sourcePort	Source port to match. If not provided, matching will be based only on IP address.	N/A
ecp.natTable[i].destinationPort	Source port will be translated to this port. Must be defined when sourcePort is provided.	N/A

Where *i* represents an index of a record in the NAT table starting from 0.

5.20. How to configure antivirus

To use antivirus scan on incoming message's payload, the external antivirus service must be running and accessible from the ECP Endpoint. ECP currently supports two antiviruses: ClamAV 0.100.0 and Symantec Protection Engine for Cloud Services 7.9.1.12.

Endpoint with enabled antivirus scan requires antivirus integration libraries ***endpoint-antivirus-scan.jar***, ***clamav-client-1.0.1.jar*** and ***SymJavaAPI-7.9.1.2.jar***. These libraries must be placed in the folder */usr/share/ecp-endpoint/webapps/ECP_MODULE/WEB-INF/lib*. Antivirus settings is configured in the Endpoint runtime configuration using properties from the following table. Finally, Endpoint must be restarted in order to start using the antivirus scanning.

Property	Description	Default value
ecp.endpoint.antivirus.antivirusEnabled	This property enables (true) or disables (false) antivirus scan.	false
ecp.endpoint.antivirus.antivirusClient	This property sets the antivirus service name used for scanning. Value must be either "clamav" (ClamAV server) or "symantec" (Symantec Protection Engine for Cloud Services server). This property is mandatory.	N/A
ecp.endpoint.antivirus.antivirusIp	IP address of the server where the external antivirus service is running. This property is mandatory.	N/A
ecp.endpoint.antivirus.antivirusPort	Network port of the external antivirus service.	3310 (ClamAV), 1344 (Symantec)
ecp.endpoint.antivirus.antivirusTimeout	Maximum time (in milliseconds) of an antivirus scan before timeout.	10000
ecp.endpoint.antivirus.quarantinePath	Path to the quarantine folder. This property is mandatory.	N/A

Endpoint uses IP address of the server defined in the configuration file to connect to the external antivirus service. If an infection is found, an infected payload is moved to the quarantine folder and the message will not be delivered.

5.21. How to restrict Jolokia access

Both ECP Endpoint and Hawtio use Jolokia API. By default, Jolokia allows connections only from localhost. Security policy of Jolokia is defined in ***jolokia-access.xml*** file in the following path:

- *Windows:*
  - *<install_path>\tomcat\webapps\ECP_MODULE\WEB-INF\classes\jolokia-access.xml*
- *Centos/RHEL:*
  - *<install_path>/webapps/ECP_MODULE/WEB-INF/classes/jolokia-access.xml*

In this file, *<remote>* section defines list of IP addresses, which can connect to the Jolokia. If the section is missing, all connections from any IP address are allowed. It is necessary to restart ECP Endpoint if the file is changed.

5.22. How to enable JMX for remote monitoring

ECP Endpoint supports monitoring by using JMX. To activate this function on ECP Endpoint, it is necessary to enable it.

5.22.1. Enabling JMX

For enabling JMX on Unix or Windows system, follow the steps bellow:

1. *Edit file located in the folder*
   - *Linux system: /etc/ecp-endpoint/jmxremote.properties*
   - *Windows system: <install_path>\jmxremote.properties*
2. *Add JMX parameters described at chapter Recommended JMX remote settings.*
3. *Save the file.*
4. *Edit system configuration file (see chapter System configuration)*
   - *set property spring.jmx.enabled=true*
5. *Save the file.*
6. *Restart ECP Endpoint.*
7. *After the restart you can connect to ECP Endpoint using JMX.*

5.23. How to restore registration process

It is possible to run Endpoint registration process for already registered Endpoint. In *Settings* page, click on *Initiate Registration* button and make a confirmation in a modal window. You will be then redirected to the first page of the registration process, and you can proceed with a registration.

Warning: All ECP Endpoint data will be removed from a database and keystores when a new registration process is confirmed.

5.24. External database configuration

These properties are required for High Availability installation or for a standalone installation with an external database:

Parameter	Description
spring.profiles.active	To enable HA mode value must be set to *ecp-ha*. Please notice that *ecp-ha* profile can be also set to enable use of an external database when running component in a standalone mode. Profile *ecp-nonha* is intended for running component in standalone mode against embedded DB.
ecp.db.url	Connection URI to external database. You may find the pattern appropriate for your database in section *External Databases*. Example: jdbc:mariadb://127.0.0.1:3306/ecp?characterEncoding=UTF-8&useSSL=false
ecp.db.username	Database user username. Example: ecp
ecp.db.password	Database user password. Example: password
spring.datasource.validation-query	This value should be set to a valid query for the selected database (see *External Databases* in ECP Installation guide). Example for the embedded DB: ‘select 1’ (without quotation marks)
ecp.cluster.manager.localInstanceID	Node identifier in the cluster. This value must be unique. Example: E1a or server’s hostname

5.25. Other configuration settings

Property	Description	Default value
ecp.keystore.location	Location of the keystore used as certificates storage. This keystore is created during the Endpoint registration.	<Endpoint data directory>/keystore.jks
ecp.keystore.password	Password to the keystore and password to the private keys inside the keystore.	password
ecp.authKeystore.location	Location of the keystore used for the authentication. This keystore contains a default private key, which is replaced during the Endpoint registration.	<Endpoint data directory>/authKeystore.jks
ecp.authKeystore.password	Password to the authentication keystore and password to the private keys inside the authentication keystore.	password
ecp.authKeystore.brokerConnectionDelayHours	A delay in hours to postpone the replacement of the authentication certificate in the JKS authentication keystore after the certificates renewal. ECP Endpoint uses the authentication certificate from the keystore to make a connection to ECP Broker, so this delay makes sure that all ECP Brokers in the network will be synchronized before ECP Endpoint will use a new certificate for a connection. The delay must be less than a value from ecp.automaticUpdate.dayBefore (30 days by default), otherwise a default value will be used.	20 * 24 hours
ecp.csrf.secret	This value is used as a secret for a creation of the CSRF token hash in GUI. It should contain random characters, like a password.	Configured during the installation
internalBroker.queuePrefix	A unique prefix, which will be added to every internal queue name belonging to Endpoint. Required when multiple different Endpoints (with different EIC codes) connect to one common Internal Broker. It is not required when nodes of only one Endpoint are connected to the Internal Broker. Otherwise, all nodes belonging to one Endpoint must use the same prefix. Allowed characters are A-Z, a-z, digits 0-9, dot .,	N/A

	underscore _, and hyphen -.	
internalBroker.baQueuePrefix	A unique prefix, which will be added to every Endpoint queue used by the business application. Useful when the business application queues should have different prefix than the internal queues. An empty value can be set, so no prefix is used. Allowed characters are A-Z, a-z, digits 0-9, dot ., underscore _, and hyphen -.	Value of internalBroker.queuePrefix

To enable batch receiving of delivered messages (GUI use case) set the value of the following property to true:

Property	Description	Default value
ecp.messagebox.showReceiveAll	This property enables receipt of all delivered messages at once by downloading a compressed file.	false

Private keys and certificates are stored as JKS files inside the database, which have password from the following property:

Property	Description	Default value
ecp.db.keystore.password	Password of JKS files with private keys and certificates stored in database. This property is mandatory.	N/A

5.25.1. How to configure connectivity check

Connectivity check use case timeout can be adjusted by the following properties:

Property	Description	Default value
ecp.endpoint.connectivityCheckAttemptTimeout	Connectivity check attempt timeout interval in milliseconds. Specifies time to wait for response from remote ECP component.	2000ms
ecp.endpoint.connectivityCheckAttemptsCount	Number of connectivity check attempts to receive tracing acknowledgements	5

5.25.2. How to configure message compression

ECP Endpoint can be configured to compress messages payload to save network bandwidth. Payload compression can be enabled by message types:

Property	Description	Default value
ecp.endpoint.compressionEnabled	To enable compression of messages set value of this property to true.	true
ecp.endpoint.messageTypesToCompress	Defines which message types are marked to be compressed (multiple values separated by comma). For type matching rules description see section *Message type matching rules*.	*
ecp.endpoint.messageTypesSkipCompress	Defines which message types are marked to skipped during compression (multiple values separated by comma). For message type matching rules description see section *Message type matching rules*.	N/A

ECP is using Java deflate (level 9) compression algorithm for compression of the data before sending them over the ECP network. This compression algorithm best fits for the text files, e.g. text file that has 10MB can be compressed to 150kB.

The differentiation between some BusinessTypes to be compressed and others not, is because of that some messages can have the binary content that does not make sense to compress (e.g. content is already compressed).

5.25.3. How to configure JMS headers validation

ECP Endpoint can be configured to validate JMS headers. This configuration is enabled in default, and it can be changes in parameter:

Property	Description	Default value
ecp.endpoint.jmsHeadersValidationEnabled	To enable / disable JMS headers validation (e.g. replyTo).	true

5.25.4. How to configure message table compression job

Default embedded database for ECP Endpoint is Apache Derby. In this database system, it may happen that message table use a lot of disk space. ECP provides compression job, which will periodically call Derby system procedure SYSCS_UTIL.SYSCS_COMPRESS_TABLE for message table. This system procedure will reclaim an unused allocated space in the table. If there is no unused allocated space, there will be no change of database disk usage after the run of the compression job. The job is disabled by default.

All outbox and inbox messaging routes are suspended until the compression job is finished. This is necessary to prevent deadlocks in the Message table if the compression and message processing would run at the same time. Once the job is finished (or the job failed in case of an exception), all suspended routes are resumed, and the message processing can continue.

ECP database compression job automatically detects whether Derby is used, and it is ignored for other database systems. It is not recommended to run this job very often; default interval is once a day. Use these properties for configuration of the compression job:

Property	Description	Default value
ecp.db.messageCompressionJobEnabled	Enable / disable database compression job.	false
ecp.db.messageCompressionInterval	Defines period how often the compression job will be executed. Value is defined as CRON expression. Default interval is once a day.	15 30 10 * * * (every day at 10:30:15)

5.25.5. How to configure connection URL parameters of ECP Brokers

When a connection to ECP Broker requires AMQPS, ECP Endpoint automatically adds required keystore and truststore parameters to the connection URL. Besides these parameters, it is also possible to configure additional custom parameters which will be added to the connection URL, for example the prefetch size.

Please refer to the Apache Qpid documentation for a list of all parameters and their description.

Use the following properties to configure the connection URL parameters:

Property	Description	Default value
ecp.brokersParameters[i].broker	The component code of ECP Broker for which the connection parameters will be applied. It is possible to use a wildcard character * to apply the parameters to all ECP Brokers. It is possible to append a wildcard character * to the partial code. Then all ECP Brokers with the same beginning will be matched. A record with the longest matching partial code will be selected. When searching which parameters should be	N/A

	used in the connection, a record with the specific component code has a higher priority than the partial component code, and a record with only the wildcard has the lowest priority.	
ecp.brokersParameters[i].parameters	The connection parameters which will be appended to the ECP Broker connection URL. Multiple parameters must be separated by & character. Each *"transport."* or *"amqp."* parameter must have the *failover.nested.* prefix.	N/A

Where *i* represents an index of a record starting from 0.

5.26. Certificates renewal

ECP Endpoint supports automatic or manual certificates renewal. Both procedures are available from the web interface of the Endpoint.

Certificates which can be renewed are: Authentication, Signing, Encryption. Registration certificate will not be renewed anymore.

- *Manual renewal – navigate to the Settings tab, scroll down to the Certificates section, and click on "Renew Manually"*
  - *Certificates can be renewed manually at any time*
  - *When performing manual renewal, certificates must be updated before the expiration of the old ones*
- *Automatic renewal – navigate to the Settings tab, scroll down to the Certificates section, and click on "Enabled" under the Automatic Renewal label*
  - *Automatic renewal job is triggered every day at 09:50, but the certificates are renewed only when there is less than 30 days remaining until their expiration.*

The certificate renewal request is approved automatically. Certificates will be shown in the Settings tab under the Certificates section.

When sending a message, ECP Endpoint will use the previous signing certificate for 20 days by default after the renewal before switching to the new certificate. This will give other Endpoints time to synchronize and get the renewed certificates. The delay can be configured by the **ecp.automaticUpdate.messagingDelayHours** property.

Property	Description	Default value
ecp.automaticUpdate.enabl	Enable / disable automatic certificate	True

ed	renewal on Endpoint.	
ecp.automaticUpdate.dayBefore	When there are less remaining days until the certificate expiration than this value, certificates will be automatically renewed.	30 days
ecp.automaticUpdate.messagingDelayHours	Sent messages will be signed by an old signing certificate until this delay will pass. This makes sure that all ECP Endpoints in the network are synchronized before the ECP Endpoint will use a renewed signing certificate.	20 * 24 hours

5.27. Backup and restore data and configuration

This chapter describes how to backup Endpoint data and configuration. If Endpoint uses external database, refer to instructions from the vendor of the external database system for database backup and restoration.

5.27.1. Windows installation

Backup these files and folders containing Endpoint data and configuration. Copy backed up files to these paths for installation restore.

- *<install_path>\authKeystore.jks*
- *<install_path>\ecp.properties*
- *<install_path>\ecp-logback.xml*
- *<install_path>\ecp-users.properties*
- *<install_path>\jmxremote.properties*
- *<install_path>\keystore.jks*
- *<install_path>\content*
- *<install_path>\db*

5.27.2. Linux installation

Backup these folders containing Endpoint data and configuration. Copy backed up files to these paths for installation restore.

- */etc/systemd/system/ecp-endpoint.service.d/env.conf*
- */etc/ecp-endpoint*
- */var/lib/ecp-endpoint**

* This corresponds to the value of dataDirectory in ***ecp.properties***. If the data directory is set differently, the backup path would be the one set in the ***ecp.properties*** file.

5.28. Configure cluster lock intervals

In case of a high availability installation, ECP Endpoint node attempts to obtain a master lock every 5 seconds by default. It sets an expiration time of the lock to be three times bigger than lock attempt interval, so it is 15 seconds by default. These values can be changed by adding these properties in the ecp.properties configuration file:

Parameter	Description	Default value
ecp.cluster.manager.scheduleInterval	Lock attempt interval	5000
ecp.cluster.manager.lockValidity	Expiration time of the lock. Must be greater than lock attempt interval.	3 * value of ecp.cluster.manager.scheduleInterval

5.29. IP change of the server

If you made an infrastructure change which resulted in the change of the IP of the application server, it is necessary to update Endpoint connection URI in the configuration of a business application or EDX component connected to the Endpoint.

5.30. Component revocation

Component Directory administrator can revoke ECP Broker or ECP Endpoint registered to their Component Directory. To revoke a component, open its detail page from the Registered Components table, and click on the “Revoke” button.

Revoked component will be removed from the component list sent to the synchronized components, so it will not be possible to send a message to a revoked component once a sender Endpoint has been synchronized with the Component Directory after the revocation.

From a revoked Endpoint, any sent message will not pass the validation step, and the reason will be logged in the application log. Note that the GUI will still allow to send a message. There will be no error message displayed, but the sent message will not be listed in the Outbox page.

Revoked Endpoint will not run the synchronization job, and its readiness actuator will show that the Endpoint is revoked.

To re-approve a revoked component, Component Directory administrator must use the “Re-approve” button in the detail page of the revoked component, accessible from the Revoked section in the Registered Components table. Then an administrator of the revoked component must manually push its configuration to the Component Directory.

5.31. Multiple ECP Endpoints with one Internal Broker

When multiple ECP Endpoints shares one Internal Broker instance, it is necessary to configure a unique queue prefix for each ECP Endpoint. This prefix will be then added to every queue name belonging to the ECP Endpoint.

Allowed characters for the queue prefix are A-Z, a-z, digits 0-9, dot ., underscore _, and hyphen -.

To configure the queue prefix, add this property to ***ecp.properties***:

> *internalBroker.queuePrefix*

If desired, a different queue prefix can be used for the queues used by the business application.

To configure a different queue prefix for the business application queues, add this property to the **ecp.properties**:

> *internalBroker.baQueuePrefix*

When no prefix should be used for the business application queues, include the property above in the configuration with an empty value.

6. Internal Broker configuration

6.1. Enable HTTPS

It is possible to enable HTTPS for the Artemis Management Console. Open ***bootstrap.xml*** in the configuration folder:

> *Linux system: /opt/eccosp-artemis/etc*

> *Windows system: <install_path>/broker/etc*

Find the *binding* element. Change *http://* to *https://* in the *uri* attribute. Then configure the following attributes:

> *keyStorePath – A path to the keystore file.*

> *keyStorePassword – A keystore password. Can be masked using ENC() syntax.*

> *sniHostCheck – Set it to false when the certificate in the keystore does not contain a host name in the Common Name or in the Subject Alternative Names. Default is true.*

> *includedTLSProtocols – TLSv1.3 is a recommended protocol.*

> *includedCipherSuites – Following ciphers are recommended: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256*

6.2. Keystore configuration

By default, Internal Broker accepts connections on 5672 port, and verifies AMQPS connections by using a provided default keystore, which is included in *keystore.jks* in the following path:

- *Windows:*
  > *<install_path>\broker\etc*
- *Linux:*
  > */opt/eccosp-artemis/etc*

It is recommended to replace the default keystore with a custom one. Either replace the *keystore.jks*, or change properties in the *acceptor* in ***broker.xml*** in the configuration folder:

- *Windows:*
  - *<install_path>\broker\etc*
- *Linux:*
  - */opt/eccosp-artemis/etc*

Change the following properties:

- *keyStorePath*
- *keyStorePassword*
- *trustStorePath*
- *trustStorePassword*

When a keystore contains multiple keypairs, it is necessary to add the *keyStoreAlias* property referencing a keypair which should be used.

When Internal Broker does not use a default keypair, its public certificate must be imported to the authKeystore.jks of all connected Endpoints, or the Endpoint *internalBroker.keystore.location* property must be changed to a keystore which already contains the public certificate. Connected Business Applications must also use the Internal Broker public certificate.

6.3. Enabling remote JMX access

Artemis supports monitoring by using JMX, which is enabled by default. It can be disabled by editing ***broker.xml*** in the configuration folder:

- *Linux system: /opt/eccosp-artemis/etc*
- *Windows system: <install_path>/broker/etc*

Add this line to the configuration:

- *<jmx-management-enabled>false</jmx-management-enabled>*

By default, JMX is enabled for a local access only. For enabling the remote JMX access, follow the steps bellow:

1. *Edit **management.xml** file located in the configuration folder*
2. *Find and uncomment this row*
   - *<connector connector-port="1099"/>*
3. *To secure the connector by using SSL, add and configure these parameters to the connector element*
   - *secured – Boolean whether the connector is secured using SSL*

- *key-store-path - The location of the keystore*
- *key-store-password - The keystore password*
- *trust-store-path - The location of the truststore*
- *trust-store-password - The truststore password*

4. *Save the file.*
5. *Restart Internal Broker.*
6. *After the restart you can remotely connect to Internal Broker using JMX.*

6.4. Passwords encryption

Passwords in Artemis configuration files can be encrypted. Any encrypted password must be wrapped in *ENC()* to be treated by Artemis as an encrypted password. Artemis provides a script for the password encryption:

- *Linux system: /opt/eccosp-artemis/bin/artemis mask '<password input>'*
- *Windows system: <install_path>/broker/bin/artemis.cmd mask "<password input>"*
- *An encrypted password will be printed. Copy all characters behind the "result:" and use them in a configuration file, wrapped in ENC()*

Encrypted passwords can be used in these configuration files in the */etc* folder:

- *broker.xml*
- *login.config*
- *bootstrap.xml*
- *management.xml*

By default, Artemis uses a deprecated default password codec. Therefore, it is recommended to use a custom password codec. Please refer to the Artemis documentation for more details.

Passwords in **artemis-users.properties** cannot be encrypted, but they can be hashed (see the next chapter).

6.5. User management

Users that can access the Artemis Management Console or establish a connection to the AMQPS acceptor are defined in **artemis-users.properties** file in the configuration folder:

- *Linux system: /opt/eccosp-artemis/etc*
- *Windows system: <install_path>/broker/etc*

For users accessing the console, it is possible to use the LDAP authentication instead (see the next chapter).

A new installation comes with one predefined *admin* user. It is strongly recommended to replace this user with your own or at least change the password. Users are stored in the following format:

- *username = password*

A new users can be added manually in the file. The Artemis script for creating a hashed password can be used:

- *Linux system: /opt/eccosp-artemis/bin/artemis mask --hash '<password input>'*
- *Windows system: <install_path>/broker/bin/artemis.cmd mask --hash "<password input>"*
- *A hashed password will be printed. Copy all characters behind the "result: 1024:" and use them in the **artemis-users.properties** file, wrapped in ENC()*

6.6. LDAP authentication

It is possible to enable LDAP (Lightweight Directory Access Protocol) authentication in the Artemis Management Console. With LDAP authentication enabled, **artemis-users.properties** file is then used only for the AMQP connections, and not for the console.

The LDAP server must be either the Active Directory server or a server supporting the standard LDAP protocol. A connection to a server can be made by LDAP or LDAPS (LDAP secured with TLS) protocol. LDAPS requires a truststore with a public certificate of the LDAP server. If LDAP server requires a client certificate authentication, a keystore with a client keypair is required.

Following steps must be done to enable LDAP authentication:

1. *Backup **bootstrap.xml** and **broker.xml** from the configuration folder.*
   - *Linux system: /opt/eccosp-artemis/etc*
   - *Windows system: <install_path>/broker/etc*
2. *In **bootstrap.xml**, rename domain in the jaas-security element from "activemq" to "activemq-properties".*

3. *Copy and replace **login.config** from <configuration folder>/ldap or from <configuration folder>/ldap/activeDirectory (when Active Directory is used) to <configuration folder>.*
4. *Configure properties in **login.config**.*

The login module in ***login.config*** is a custom ECCo SP module, which extends the Artemis login module with a possibility to use LDAPS connection. It also changes the handling of the Active Directory bind user; Instead of configuring the bind user and password, values from the login form are used for the binding.

These properties are common for both Active Directory and standard LDAP:

Property	Description	Default value
initialContextFactory	The initial context factory used to connect to LDAP. It must always be set to the default value.	com.sun.jndi.ldap.LdapCtxFactory
connectionUrl	URL and a port number of the LDAP server. It must start with “ldap://” or “ldaps://”. If it starts with “ldaps://”, a connection will be secured with TLS. You can optionally qualify this URL by adding a forward slash /, followed by the DN (distinguished name) of a particular node in the directory tree.	ldaps://127.0.0.1:636
connectionProtocol	Change it to a blank string when LDAPS is not used.	ssl
connectionTimeout	A request will timeout when a connection to the LDAP server is not established within this time limit in milliseconds.	5000
readTimeout	A request will timeout when the LDAP server will not return a result within this time limit in milliseconds, after a connection has been established.	5000
authentication	Specifies the authentication method used when binding to the LDAP server. Can be set to *simple* (username and password) or *none* (anonymous).	simple
verifyHost	Disables hostname verification. Set it to *false* when certificates do not contain LDAP server hostname in the Common Name or Subject Alternative Names.	true
truststoreLocation	A path to the truststore file with a public certificate of the LDAP server.	/opt/ecp-broker/broker/

		etc/ldap-truststore.jks
truststorePassword	A truststore password.	password
keystoreLocation	A path to the keystore file with a client private keypair.	/opt/ecp-broker/broker/etc/ldap-keystore.jks
keystorePassword	A keystore password.	password
keyPassword	A password to a keypair in the keystore.	password
userSearchSubtree	Specify the search depth for user entries, relative to the node specified by *userBase*. *False* indicates it will try to match one of the child entries of the *userBase* node. *True* indicates it will try to match any entry belonging to the subtree of the *userBase* node.	false

This property can be changed for Active Directory:

Property	Description	Default value
connectionUsername	This property is not used with Active Directory, because the username from the login form is used as a bind user.	ignored
connectionPassword	This property is not used with Active Directory, because the password from the login form is used as a bind password.	ignored
userBase	Selects a particular subtree to search for user entries.	cn=users,dc=domain,dc=com
userSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *userBase*. The *{0}* substring is substituted by the username extracted from the incoming client credentials passed in the login form. Replace *domain.com* to match a domain from *userBase.*	(&(objectClass=user)(userPrincipalName={0}@domain.com))
roleName	Specifies the attribute type of the role entry that contains the name of the role/group (e.g. C, O, OU, CN etc.). If you omit this	CN

	option, the full DN (distinguished name) of the role is used.	
userRoleName	Specifies the name of the multi-valued attribute of the user entry that contains a list of role names for the user (where the role names are interpreted as group names by the broker's authorization plug-in). If you omit this option, no role names are extracted from the user entry.	memberof
ignorePartialResultException	Should be always set to *true*.	true

These properties must be changed for a standard LDAP:

Property	Description	Default value
connectionUsername	The DN of the user that opens the connection to the LDAP server.	uid=admin,ou=system
connectionPassword	The password that matches the DN from *connectionUsername*. In the LDAP server, the password is normally stored as a *userPassword* attribute in the corresponding directory entry.	password
userSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *userBase*. The *{0}* substring is substituted by the username extracted from the incoming client credentials passed in the login form.	(uid={0})
roleName	Specifies the attribute type of the role entry that contains the name of the role/group (e.g. C, O, OU, CN etc.). If you omit this option, the full DN (distinguished name) of the role is used.	cn
roleBase	If you want to store role data directly in the LDAP server, you can use a combination of role options (*roleBase*, *roleSearchMatching*, *roleSearchSubtree*, and *roleName*) as an alternative to (or in addition to) specifying the *userRoleName* option. This option selects a particular subtree DIT to search for role/group entries. The subtree is specified by a DN, which specifies the base node of	ou=groups,dc=domain,dc=com

	the subtree.	
roleSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *roleBase*. This works in a similar manner to the *userSearchMatching* option, except that it supports two substitution strings, as follows: {0} - substitutes the full DN of the matched user entry (that is, the result of the user search). For example, for the user *jdoe*, the substituted string could be *uid=jdoe,ou=User,ou=ActiveMQ,ou=system*. {1} - substitutes the received username. For example, *jdoe*. Leaving this property unset disables role searching, and the role information must come from *userRoleName*.	(uniqueMember ={0})
roleSearchSubtree	Specifies the search depth for role entries, relative to the node specified by *roleBase. False* indicates it will try to match one of the child entries of the *roleBase* node. *True* indicates it will try to match any entry belonging to the subtree of the *roleBase* node.	false

6.7. How to enable direct communication

To support a direct messaging in the connect ECP Endpoint, a new AMQPS acceptor must be configured in ***broker.xml*** file in the following location:

- *For Windows OS: <install_path>/broker/etc*
- *For Linux OS: /opt/eccosp-artemis/etc*

Find the *acceptors* element and create another AMQPS acceptor. It must use a different port than the existing port (for example to 5671), and the referenced keystore must be accessible by the Internal Broker (like *authKeystore.jks* of ECP Endpoint). Also add the *needClientAuth=true* parameter to the acceptor URL.

Find the *broker-plugins* element and add these lines inside the element:

<broker-plugin class-name="eu.entsoe.ecp.artemis.plugin.auth.direct.EcpBaseDirectPlugin">

```
    <property key="ecpComponentCode" value="CHANGE-ME-COMPONENT-CODE" />
</broker-plugin>
```

Replace *CHANGE-ME-COMPONENT-CODE* with a component code of the connected ECP Endpoint.

Open ***bootstrap.xml*** file in the following location:

- *For Windows OS: <install_path>/broker/etc*
- *For Linux OS: /opt/eccosp-artemis/etc*

Replace the *jaas-security* element with these lines:

```
<security-manager class-name="eu.entsoe.ecp.artemis.plugin.auth.direct.EcpAuthorizationDirectPlugin">
    <property key="ecpComponentCode" value="CHANGE-ME-COMPONENT-CODE" />
</security-manager>
```

Replace *CHANGE-ME-COMPONENT-CODE* with a component code of the connected ECP Endpoint.

6.8. How to connect BA over AMQPS

A connection between a business application and the Internal Broker over AMQPS must be authenticated by user and password, which must exist in the Internal Broker ***artemis-user.properties*** file in the following location:

- *For Windows OS: <install_path>/broker/etc*
- *For Linux OS: /opt/eccosp-artemis/etc*

AMQPS requires a truststore file with the public certificate or the Certificate Authorities, which are used by Internal Broker. The certificate can be exported from the keystore file referenced in *acceptor* element in the *broker.xml* file.

A client certificate authentication is disabled by default in the Internal Broker configuration. When it is enabled, a keystore file with a private key for an identification of the business application is required.

The following properties must be added to the AMQPS connection URI when required:

Property	Description
transport.trustStoreLocation	Path to a truststore file.
transport.trustStorePassword	Password of a truststore file.

transport.keyStoreLocation	(Optional) Path to a keystore file.
transport.keyStorePassword	(Optional) Password of a keystore file.
transport.keyAlias	(Optional) Alias of the private key in a keystore file. Can be configured to select a specific key when a keystore contains multiple private keys.
transport.verifyHost	(Optional) Set the value to false when a certificate in a truststore does not include a server name in the Common Name or in the Subject Alternative Name, otherwise set it to true. Default is true.

6.9. How to disable AMQPS and authentication

AMQPS is enabled by default. To disable it, open ***broker.xml*** file at the following location:

- *For Windows OS: <install_path>/broker/etc*
- *For Linux OS: /opt/eccosp-artemis/etc*

Find the *acceptors* element and set the *sslEnabled* parameter in the acceptor URL to false. ECP Endpoint connected to this acceptor must also disable AMQPS on its side. See the *How to disable AMQPS* subchapter in the *ECP Endpoint Administration* chapter for more details.

To completely disable authentication, add this line inside the *<core>* element in ***broker.xml***:

- *<security-enabled>false</security-enabled>*

6.10. Plugins properties

Internal Broker can use several ECP plugins, each of them has configurable properties.

6.10.1. EccospAuditLoggerPlugin

EccospAuditLoggerPlugin configuration is in the */opt/ecp-broker/broker/etc/broker.xml* file, inside the *<broker-plugins>* element.

Property	Description	Default value

ecpComponentCode	ECP component code, which will be written in the logs. It can be set to any value – for example a code of the connected ECP Endpoint.	CHANGE-ME-COMPONENT-CODE
ecpComponentType	ECP component type, which will be written in the logs. Default value should be kept.	ECCo SP Artemis Internal Broker

6.10.2. EcpBaseDirectPlugin

EcpBaseDirectPlugin configuration is in the */opt/ecp-broker/broker/etc/broker.xml* file, inside the *<broker-plugins>* element. It is not configured by default; It must be added manually when enabling the direct messaging.

Property	Description	Default value
ecpComponentCode	ECP component code of the connected ECP Endpoint.	N/A
dlq	Name of the Dead Letter Queue, where the undeliverable messages will be sent.	DLQ
clusterUser	Username used for a validation of a message sent from another node in Artemis cluster. Its value should be the same as the value of the *<cluster-user>* element in broker.xml. See the chapter *Plugins user credentials* for more details.	N/A

6.10.3. EcpAuthorizationDirectPlugin

EcpAuthorizationDirectPlugin configuration is in the */opt/ecp-broker/broker/etc/bootstrap.xml* file, inside the *<security-manager>* element. It is not configured by default; It must be added manually when enabling the direct messaging.

Property	Description	Default value
ecpComponentCode	Code of the ECP Component that uses this internal broker for messaging.	N/A

clusterUser	Username used for a validation of a message sent from another node in Artemis cluster, management message, or Artemis CLI command request. When the Internal Broker is in Artemis cluster, the value should be the same as the value of the *<cluster-user>* element in broker.xml. See the chapter *Plugins user credentials* for more details.	N/A
clusterPassword	Password used for a validation of a message sent from another node in Artemis cluster, management message, or Artemis CLI command request. When the Internal Broker is in Artemis cluster, the value should be the same as the value of the *<cluster-user>* element in broker.xml. When encrypted, it must be wrapped inside *ENC(…)*. See the chapter *Plugins user credentials* for more details.	N/A
passwordCodec	An Artemis password codec used for the encryption of the *clusterPassword.* When not defined, a default Artemis codec is used. See the chapter *Plugins user credentials* for more details.	N/A

6.11. Plugins user credentials

ECP plugins for the direct messaging in Internal Broker validate and authenticate messages and connections by loading an ECP component code from the authentication certificate used for a connection from ECP component to Internal Broker.

When the direct messaging is enabled and when a message is sent to Internal Broker from another Internal Broker node in the Artemis cluster (due to the load balancing or redistribution), when a management message is received (using Artemis CORE protocol), or when a request from Artemis CLI command is received, then the ECP certificate authentication will not work; There is no certificate sent in these cases, because they expect the user and password authentication instead.

To allow user and password authentication, it is possible to configure *clusterUser* and *clusterPassword* plugin properties. When defined, Internal Broker will try first to perform user and password authentication before the certificate authentication. See the *Plugins properties* chapter for details on how to configure these properties.

Only one user and password is allowed to be configured in the plugin properties. When ECP Broker is in Artemis cluster, then *clusterUser* and *clusterPassword* must use the same values as the *<cluster-user>* and *<cluster-password>* elements from the cluster configuration in the **broker.xml** file.

When a password in *clusterPassword* is encrypted, it must be written in the Artemis *ENC(…)* syntax. When a custom password codec was used, it must be configured in the *passwordCodec* plugin property. A custom key used for the encryption can be defined either in *artemis.default.sensitive.string.codec.key* Java property, or in *ARTEMIS_DEFAULT_SENSITIVE_STRING_CODEC_KEY* environment property.

6.12. Backup and restore data and configuration

This chapter describes how to backup Internal Broker data and configuration.

6.12.1. Windows installation

Backup these files and folders containing Internal Broker data and configuration. Copy backed up files to these paths for installation restore.

- *<install_path>\broker\etc*
- *<install_path>\data*

6.12.2. Linux installation

Backup these folders containing Internal Broker data and configuration. Copy backed up files to these paths for installation restore.

- */opt/eccosp-artemis/etc*

- */opt/eccosp-artemis/data*

7. ECP Component Directory Administration

This section describes configurations available in the Component Directory.

7.1. User authentication

To prevent an unauthorized access, Component Directory requires users to log in by default. To disable user authentication, find the property *spring.profiles.active* in the ***ecp.properties*** file and add profile *disable-user-auth* to the list of profiles. It is strongly recommended to use HTTPS connection when user authentication is enabled.

A list of allowed users is defined in ***ecp-users.properties*** file in the configuration directory. A new installation comes with one predefined user: admin/password. It is strongly recommended to replace this user with your own or at least change the password to a more secure one.

The pattern is:

> *ecp.directory.users[index].login=user*

> *ecp.directory.users[index].password=password*

The index must be a whole number from a sequence starting with zero. Example:

> *ecp.directory.users[0].login=janedoe*

> *ecp.directory.users[0].password=securepassword*

> *ecp.directory.users[1].login=johndoe*

> *ecp.directory.users[1].password=secret*

7.1.1. Monitoring Authentication

By default, an access to all actuator endpoints exposed on */ECP_MODULE/actuator/** URL is protected by the user authentication. It is possible to disable the authentication for individual endpoints by adding the endpoint name to the comma-separated list in *ecp.disabledAuthenticationActuatorEndpoints* property in the ***ecp.properties*** file.

The *management.endpoints.web.exposure.include* property contains the list of names of the exposed actuator endpoints.

7.1.2. Passwords in Hash

ECP Endpoint supports password stored in the SHA512 encryption with a salt prefix.

Example:

ecp.directory.users[index].password=6<salt_prefix>$<hashed_password>

7.1.2.1. How to generate hashed password with salt

Prerequisites: installed command line tool 'openssl' which is available for all OS.

The hashed password is generated with command:

openssl passwd -6 -salt <salt>

- *option -6 indicates algorithm SHA512*
- *option -salt indicates selected salt prefix*

Result of generating hash in this template:

- *6<salt_prefix>$<hashed_password>*

For example:

 # openssl passwd -6 -salt test

Password: <enter password>

6test$zW0OdEUeeqYAEUmnXZ3CcdTnF867qiC1306IvgaUfV7MqmQ0lBIaxFBkpxpaEqO5eLJrDe2F8PaZXYM0eTP04.

7.2. How to configure unsuccessful login attempts

To ensure better security, Component Directory supports a configurable limit of the number of unsuccessful login attempts. After the limit is reached, login will be locked for a period of time.

Property	Description	Default value
ecp.loginLock.attempts	Limit of unsuccessful login attempts	3
ecp.loginLock.lockTime	Lock time in seconds	900

7.3. HTTPS connector configuration

Navigate to your installation directory and locate the following file:

- */usr/share/ecp-directory/conf/server.xml*

In Component Directory there is HTTPS connector enabled and configured by default. If you want to change the parameters of HTTPS , you should edit one of the parameters listed in file of **server.xml**, where you can find similar code to:

```
<Connector
    port="8443"
    protocol="eu.entsoe.ecp.connector.protocol.EcpDirectoryHttp11NioProtocol"
    sslImplementationName="org.apache.tomcat.util.net.jsse.JSSEImplementation"
    maxThreads="150" SSLEnabled="true" maxParameterCount="1000" >
    <SSLHostConfig
        protocols="TLSv1.3" truststoreFile="/var/lib/ecp-directory/authKeystore.jks" truststorePassword="password" certificateVerification="optional" ciphers="TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SHA256,TLS_AES_128_GCM_SHA256" >
        <Certificate certificateKeystoreFile="/var/lib/ecp-directory/authKeystore.jks" certificateKeystorePassword="password"
        certificateKeyAlias="ecp_module_auth" type="RSA" />
    </SSLHostConfig>
</Connector>
```

As a keystore file can be used any trusted keystore pair including the authentication one present in ECP (that is the default filled up parameter). There is a default keypair present in the authentication keystore, which is replaced by the ECP authentication keypair during the registration process. The default keypair is valid until 21.02.2026.

Parameter *ciphers* specifies which ciphers can be used for HTTPS connection. If not specified, the JVM default ciphers from protocols specified in *protocols* are used.

7.4. Certificates renewal

ECP Component Directory supports automatic and manual certificates renewal.

Automatic certificates renewal is enabled by default in Component Directory. This feature can be disabled and configured with the following properties in the configuration file:

Property	Description	Default value
ecp.automaticUpdate.enabled	Enable / disable automatic certificate renewal on Component Directory	True
ecp.automaticUpdate.checkInterval	Defines period how often the job will be executed. Value is defined as CRON expression. Default interval is every day at 9:40.	0 40 9 * * * (every day at 9:40)
ecp.automaticUpdate.dayBefore	When there are fewer remaining days until the certificate expiration than this value, certificates will be automatically renewed.	30

In default configuration, certificate renewal job starts every day at 9:40. If there is less than 30 days remaining until the certificates' (Registration, Authentication) expiration, they will be automatically renewed. New certificates will have Valid From time set to one hour in the past.

After the automatic certificate renewal, Apache Tomcat will automatically update its runtime SSL context to use the new Authentication certificate.

When performing manual renewal, certificates must be updated before the expiration of the old ones. To update the certificates, navigate to the Settings page and use the *Import Local Certificate* to upload a keystore with the new Global and Integrated certificate. After the upload, a new Authentication signed by the new Integrated certificate will be created.

After the manual certificated renewal, the restart of Component Directory will be necessary.

7.4.1. Certificate Authorities Update

The process of updating the Certificate Authorities is described in the *ECP System Design*, chapter *5.4.4. Certificate Authorities Update.*

7.5. How to enable JMX for remote monitoring

ECP Component Directory supports monitoring by using JMX. To activate this function, it is necessary to enable it.

7.5.1. Enabling JMX

For enabling JMX, follow the steps bellow:

1. *Edit the file located in the folder /etc/ecp-directory/jmxremote.properties*
2. *Add JMX parameters described at chapter Recommended JMX remote settings.*
3. *Save the file.*
4. *Edit system configuration file (see chapter System configuration)*
   - *set property spring.jmx.enabled=true*
5. *Save the file.*
6. *Restart ECP Component Directory.*
7. *After the restart you can connect to ECP Component Directory using JMX.*

7.6. Central Message Paths Management

Instead of managing message paths for individual Endpoint, it is possible to add or remove message paths centrally through the Component Directory GUI. Endpoint periodically run message path synchronization job with its home Component Directory, which updates message paths on Endpoint.

7.6.1. How to add a new centrally managed message path

Open the Component Directory web interface and navigate to Components page. Open Central Management Paths tab in Registered components section and click the New Path button. A new path dialog box will open.

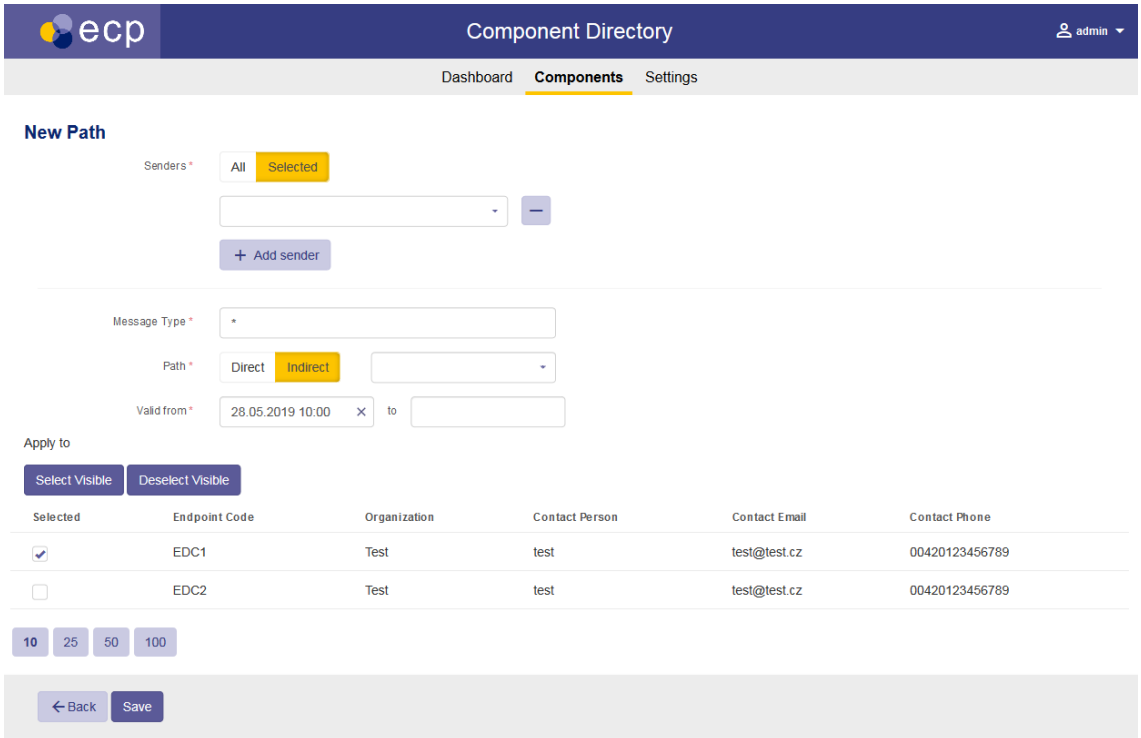

Figure 2: New Central Message Path

Indirect message paths can be created for existing intermediate components (typically ECP broker), which must allow communication with endpoints. See also configuration of broker filter: How to configure component filter.

Direct message paths can be created only when direct messaging is enabled on endpoints. For instructions on how to enable direct messaging on endpoint see How to enable direct communication.

Message path message type support's the definition of message type with a wild card character. For message path matching rules description see also chapter *Message type matching rules*.

New centrally managed message path is saved to the Component Directory Message Path Store. Centrally managed message paths from the Component Directory Message Path Store are accessible through authenticated REST API.

Endpoints periodically run message paths synchronization job with their home Component Directory. The new centrally managed message paths are validated for conflicts with the local existing message. If at least one conflict occurs the new centrally managed message path cannot be stored. For more information, please see chapter *Message path conflict resolution*. Valid centrally managed message paths are applied to the Endpoint message paths and becomes active Endpoint message paths distributed over ECP network through Component Directory Data (using *pushConfiguration* REST API).

Centrally managed message path can be deleted in its detail page on Component Directory. Centrally managed message path is deleted for all Endpoints. Change is distributed to all local Endpoints via Component Directory Message Path Store REST API – using same communication patter as for creation of the new Centrally managed message path described above.

7.7. Central Management of TTL per Message Type

Instead of managing TTL per message type for individual Endpoint, it is possible to add or remove message types centrally through the Component Directory GUI. Endpoint periodically run message types synchronization job with its home Component Directory, which updates message types on Endpoint.

To add new message type with TTL, navigate to Components page and open Message Types tab in Registered components section. By clicking on "+ New Message Type" button, a modal window will open with fields for Message Type and Time to live. Time to live value must contain at least one number with time unit for days, hours, minutes, or seconds. Examples of Time to live values: 2d 12h 30m 15s, 5h 20m, 7d

New Message Type must not already exist in the table. Existing message types can be edited or deleted from the table. If any message type already exists on Endpoint, Endpoint message type takes precedence over message type from Component Directory.

7.7.1. Message Paths Conflict Resolution

To prevent message paths conflicts, only Endpoint can push its message paths to the Component Directory. This means that Centrally Managed Message Paths received from Component Directory must be validated by the Endpoint.

- *Two message paths (centrally managed / local) are equals when their message type, senders' restrictions, path definition and validity are equals. Such message paths are not considered to be conflicting.*
- *Two message paths are in conflict when they have same message type and path definition, but senders' restrictions or validity are different.*
  - *When centrally managed and local message paths are in conflict, local message path takes precedence over centrally managed path.*
- *Receival of the Centrally managed message paths can be disabled by the Endpoint configuration settings.*
- *Endpoint uses only its primary (home) Component Directory for centrally managed message paths management.*

7.8. Registration keystores with specific component code

By default, any component can send registration request with generic registration keystore downloaded from Component Directory. Component Directory can be configured to create and accept registration keystores with specific component code. In that case, table “Registration Certificates” is shown on the Settings page. Component Directory Administrator can generate new registration certificate with specified component code and date of expiration. New registration certificate is stored in Component Directory repository and added into the table. From the table, registration keystore with the certificate can be downloaded or certificate detail page can be opened. It is possible to delete a certificate from Component Directory repository on certificate detail page, which prevents all components with this certificate to register to Component Directory.

Component code is not mandatory, so it is possible to create a certificate, which can be used by all components for registration. If certificate does not have a component code, then value “NOT_SPECIFIED” is shown in the table. It is not possible to add a certificate with component code already existing in one of the certificates. It is necessary to delete the certificate and then create a new one.

If the generation of the registration keystore with specific component code is enabled, it is not possible to download a generic registration keystore of Component Directory. Generic registration keystore which have been issued before is no longer accepted by Component Directory and cannot be used for registration process.

When Component Directory receives a registration request, a registration certificate from the request is compared with registration certificates stored in the repository of the Component Directory. Registration of the component will proceed only if a valid matching certificate is found and a component code from the request matches the component code from the certificate (only if certificate contains a component code).

Registration using keystores with specific component code can be enabled with the following property in the configuration file:

Property	Description	Default value
ecp.directory.registration.newCertificate.specificCodeEnabled	Enable / disable registration using keystores with specific component code	False

7.9. How to configure synchronization with remote Component Directories

Configuration of synchronization with remote Component Directories is done in GUI of Component Directory – page Components, section Other component directories. In this section, there is a list of remote Component Directories. Local Component Directory will synchronize with these CDs by periodically executed synchronization job. Synchronization interval is configured by the following property:

Property	Description	Default value
ecp.directory.component.directorySynchronizationInterval	Synchronization interval with other component directories. Standard CRON expression.	0 */10 * * * * (every tenth minute)

Component Directory Administrator can also initiate synchronization with all CDs or specified CD manually from GUI. GUI shows if last synchronization attempt was successful.

If the other Component Directory trusts different Root Certificate Authority, it is necessary to import CA keystore from the other CD as Network certificate. See chapter *Multiple Root Certificate Authorities* for details.

List of remote Component Directories can be changed by importing text file with records of CDs and synchronization setting (button Import List). Import of a new file will replace current configuration of remote CDs. To make change of a current configuration, export current configuration by clicking on Export List button, make changes in exported file and import it back.

Configuration file is a regular text file with space separated values as defined below (note that first row is header shown only for the sake of clarity - it is not supported by the configuration import). Each line in the file represents one remote Component Directory record.

Unique CD identifier	CD type	Sync mode	Allowed CDs	CD URLs
CODE	CENTRAL / COMMON	ONE_WAY / TWO_WAY / PASSIVE	CD1, CD2	URLs

- *CODE – unique identification code of the remote CD*
- *CD type*
  - *CENTRAL – Component Directory Synchronizer (CDS) - federated component directory type, provides content of multiple component directories*
  - *COMMON type of component directory providing its own content*

- *Allowed CDs - optional value, which will represent list of Component Directories, which are allowed to communicate with the CD of the current entry. This list will be a comma-separated list of components codes. When there is no value, the CD can communicate with all Component Directories connected to CDS. The list can be filled in only on CDS, on the CDS is done the filtering of available components for the specific Component Directory – detailed explanation in chapter 6.8.1 Group of ECP Networks*
- *Sync mode*
  - > *TWO_WAY synchronization mode where the configured directory pushes its own content to the remote directory specified by CODE actively together with pulling content of the remote directory. Can be enabled only on one synchronization side.*
  - > *ONE_WAY synchronization mode where the configured directory asks remote directory specified the CODE for its content. Can be enabled on both synchronization sides.*
  - > *PASSIVE synchronization mode where the configured directory allows the remote directory specified by the CODE to access / modify directory data. This mode has authentication meaning and will be used typically as a counterpart configuration for TWO_WAY or ONE_WAY sync.*

Note that component directory synchronized with central CD usually with TWO_WAY sync mode.

Example of the TWO_WAY synchronization configuration is below.

This configuration will be set on component directory CD1:

> *CD2 CENTRAL TWO_WAY https://192.168.82.1 https://192.168.82.2*

CD2 setting will be than set as follows:

> *CD1 CENTRAL PASSIVE https://192.168.82.3 https://192.168.82.4*

If a remote Component Directory is removed from the synchronization list, all its components will be removed from the content of the local Component Directory. If a removed Component Directory was a CDS, all Component Directories and their components, which were provided through the removed CDS, will be removed from the content of the local Component Directory.

7.9.1. Group of ECP networks

Example of the configuration of CDS with a restricted communication between connected Component Directories:

> *CD1 COMMON PASSIVE CD2,CD4 https://192.168.82.1*

> *CD2 COMMON PASSIVE CD1,CD4 https://192.168.82.2*

- *CD3 COMMON PASSIVE CD4 https://192.168.82.3*
- *CD4 COMMON PASSIVE https://192.168.82.4*

In the example, CD1 and CD2 can communicate between each other and with CD4, CD3 can communicate only with CD4 and CD4 can communicate with all Component Directories. Note that an administrator should make sure that communication between two Component Directories is allowed from both sides.

The communication will be restricted by filtering the response to the synchronization request. The response will contain only the content from those Component Directories, which are allowed to communicate with the Component Directory, which called the synchronization request.

When using CDS approach, the ECP Component Directory, which is CDS, must configure the following parameter. Enabling of this parameter will result in the following changes in CD functionality:

- *Any registration request over REST API is rejected*
- *Request for export or import of synchronized CDs configuration over REST API is rejected*
- *Registration keystore download request over REST API is rejected*
- *"Requests to Approve" and "Registered components" GUI sections from the Components tab are not displayed*
- *"Export List" and "Import List" buttons from the "Other component directories" section in Components tab are not displayed, changes of configuration are allowed only via GUI*
- *"Download Registration Keystore" button from the Settings tab is not displayed*
- *Four-eye control of synchronized CDs configuration is enabled*
- *"Pending edits" and "Pending removals" tables are displayed in the Components tab*

Property	Description	Default value
ecp.directory.isCDS	This property enables (true) or disables (false) CDS mode.	false

It is mandatory for CDS to be managed by at least two administrators, because there is a four-eye control of synchronized CDs configuration. When CDS administrator edits or removes synchronized CD, a new record is created in "Pending edits" or "Pending removals" table. Change of the configuration can be accepted only by a different CDS administrator. After the change is accepted, a record is removed from the pending table and the corresponding CD record is updated or deleted from the "Other component directories" table.

Security note:

By default, the user password for accessing GUI is “password”. It is recommended to change it to secure password – chapter 6.1. User authentication.

7.9.2. Configuration of synchronization via GUI

User can manage remote Component Directories via use case “Other component directories” in Components page in CD GUI. The remote Component Directory is possible “Add”, “Edit” add “Remove”. “Allowed CDs” field will be active only when “CD type” is set to COMMON and “Sync mode” is set to PASSIVE.

7.10. Cleaning job for expired specific registration certificates

It is possible to enable cleaning job, which will run periodically and removes all expired specific registration certificates from a database. Therefore, it will no longer be possible to download expired registration certificates from GUI. Cleaning job can also be configured in a way, that expired certificates will not be removed immediately, but only when number of days after expiration will reach configured value. Following properties are used for the configuration of the cleaning job:

Property	Description	Default value
ecp.specificCertificatesCleaning.enabled	Enable / disable cleaning job for expired specific registration certificates.	false
ecp.specificCertificatesCleaning.interval	Defines period how often the cleaning job will be executed. Value is defined as CRON expression.	0 20 9 * * * (every day at 9:20)
ecp.specificCertificatesCleaning.offset	Expired certificate will not be removed until number of days after its expiration will reach this value.	0

7.11. How to restrict GUI access

By default, Component Directory GUI is accessible to all hosts which can connect to the application server. Component Directory Administrator can create a whitelist of IP addresses in ecp-directory.properties configuration file. Whitelist affects only GUI access; it does not limit access to Component Directory REST API.

Access to CD GUI will be denied to a host which IP address does not match the whitelist. An error page will be displayed with an information message about the restricted access.

Whitelist can combine specific IPs and subnets. Both IPv4 and IPv6 are supported. Use these properties to configure CD GUI access:

Property	Description	Default value
ecp.directory.restrictAccess.enabled	This property enables (true) or disables (false) GUI access restriction.	false
ecp.directory.restrictAccess.allowed	Comma-separated list of IPs or subnets, which can connect to CD GUI. Example: 172.16.0.0/12, 172.17.1.15, 192.168.0.0/16	N/A

7.12. How to configure components expiration

All component records in the Component Directory have limited time to live (TTL) period. Each client updates the TTL during the synchronization job.

When a TTL of a Component Directory expires, then no message can be sent to and from a component registered within the Component Directory until a new synchronization succeeds.

Default TTL is 28 days. It can be configured by the following property:

Property	Description	Default value
ecp.directory.ttl	Synchronized components lifetime in milliseconds. Must be non-negative whole number.	2 419 200 000 (28 days)

7.13. External database configuration

These properties are required for High Availability installation or for a standalone installation with an external database:

Parameter	Description

spring.profiles.active	To enable HA mode value must be set to *ecp-ha*. Please notice that *ecp-ha* profile can be also set to enable use of an external database when running component in standalone mode. Profile *ecp-nonha* is intended for running component in standalone mode against embedded DB.
ecp.db.url	Connection URI to external database. Example: jdbc:mariadb://127.0.0.1:3306/ecp-directory?characterEncoding=UTF-8&useSSL=false
ecp.db.username	Database user username. Example: ecp-directory
ecp.db.password	Database user password. Example: password
spring.datasource.validation-query	This value should be set to a valid query for the selected database (see *External Databases* in ECP Installation guide). Example for the embedded DB: ‘select 1’ (without quotation marks)
ecp.cluster.manager.localInstanceID	Node identifier in the cluster. This value must be unique. Example: server’s hostname

7.14. Another configuration

Property	Description	Default value
ecp.directory.registration.newCertificate.validity	Validity length of issued certificates in days. Must be non-negative whole number.	365
ecp.directory.component.directorySynchronizationInterval	Synchronization interval with other component directories. Standard CRON expression.	0 */10 * * * * (every tenth minute)
ecp.urls	URLs of this Component Directory. This value is set during the registration process	No default value
ecp.keystore.location	Location of the keystore used as certificates storage. This keystore is created during the Component Directory registration.	<CD data directory>/keystore.jks
ecp.keystore.password	Password to the keystore and password to the private keys inside the keystore.	password
ecp.authKeystore.location	Location of the keystore used for the authentication. This keystore contains a default private key, which is replaced during the Component Directory registration.	<CD data directory>/authKeystore.jks
ecp.authKeystore.password	Password to the authentication keystore and password to the private keys inside the authentication keystore.	password
ecp.db.keystore.password	Password of JKS files with private keys and certificates stored in database. This property is mandatory.	N/A
ecp.directory.regKeystore.password	Password of registration keystore downloaded via GUI. This property is mandatory.	N/A
ecp.directory.caKeystore.password	Password of CA keystore downloaded via GUI. This property is mandatory.	N/A
ecp.metricsSyncThreshold	Threshold in minutes used in the metrics for not synchronized components. Components are counted in the metrics if their last synchronization time is in the past for more than this value.	10
ecp.csrf.secret	This value is used as a secret for a creation of the CSRF token hash in GUI. It should contain random characters, like a password.	Configured during the installation

7.15. Backup and restore data and configuration

This chapter describes how to backup Component Directory data and configuration. If Component Directory uses an external database, refer to instructions from the vendor of the external database system for database backup and restoration.

Backup these folders containing Component Directory data and configuration. Copy backed up files to these paths for installation restore.

- */etc/systemd/system/ecp-directory.service.d/env.conf*
- */etc/ecp-directory*
- */var/lib/ecp-directory**

* This corresponds to the value of dataDirectory in ***ecp-directory.properties***. If the data directory is set differently, the backup path would be the one set in the ***ecp-directory.properties*** file.

7.16. How to create CA keystore

For testing purposes, it is possible to create the Certificate Authority keystore with self-signed global and integrated certificates.

1. *Download KeyStore Explorer tool from https://keystore-explorer.org and start it*
2. *Click on File – New and select JKS type*
3. *Click on Tools – Generate Key Pair*
4. *Select RSA with at least of 2048 Key Size*
5. *Select Version 3, SHA-256 with RSA, set years of validity and click Apply*
6. *Click on the button next to Name field and fill the name fields*
7. *Click on Add extensions and then use the plus button to add these extensions:*
   - *Authority Key Identifier - Key Identifier 160-bit hash*
   - *Basic Constraints with Critical Extension flag - mark Subject is a CA*
   - *Key Usage - select Certificate Signing, CRL Sign, Digital Certificate*
   - *Subject Key Identifier - Key Identifier 160-bit Hash*
8. *Click OK twice*
9. *Enter global as alias*
10. *Enter password*

Creation of the integrated certificate:

1. *Right click on global certificate and select Sign - Sign New Key Pair*
2. *Select RSA with at least of 2048 Key Size*
3. *Select Version 3, SHA-256 with RSA, set years of validity and click Apply*
4. *Click on the button next to Name field and fill the name fields - Common Name must be the vCode of the ECP CD*
5. *Click on Add extensions and then use the plus button to add these extensions:*
   - *Authority Key Identifier - Key Identifier 160-bit hash*
   - *Basic Constraints with Critical Extension flag - mark Subject is a CA*
   - *Key Usage - select Certificate Signing, CRL Sign, Digital Certificate*
   - *Subject Key Identifier - Key Identifier 160-bit Hash*
6. *Click OK twice*
7. *Enter integrated as alias*
8. *Enter password*

Save the created keystore. This keystore can be later used for generating certificates for another ECP CDs.

1. *Right click on global certificate and select Export – Export Certificate Chain*
2. *Select RSA with at least of 2048 Key Size*
3. *Select Head Only, X.509, PEM, save it as* ***global.cer***
4. *Right click on integrated certificate and select Export – Export Key Pair*
5. *Select RSA with at least of 2048 Key Size*
6. *Select PKCS#12, save it as integrated.p12*

Compilation of registration keystore for ECP CD:

1. *Click on File – New and select JKS type*
2. *Click on Tools – Import Trusted Certificate and select* ***global.cer*** *file*
3. *Enter global as alias*
4. *Click on Tools – Import Key Pair, select PKCS#12 and select* ***integrated.p12*** *file*
5. *Enter integrated as alias and enter password*
6. *Save the created keystore*

Use the created keystore for the registration of the Component Directory.

8. ECP Broker Administration

8.1. How to change broker configuration

Broker configuration can be changed manually by editing the broker properties file. See Editing the configuration for instructions how to properly edit the properties file.

For Broker configuration update, the following steps should be performed:

1. *Locate /opt/ecp-broker/config/**broker.properties** file*
2. *Edit **broker.properties** file content*
3. *Push Broker configuration to ECP Component Directory*
4. *Restart Broker*

For instructions on how to push Broker configuration and restart Broker see the ECP Installation Guide.

Note that for broker deployed in HA setup configuration, files located at each Broker node must be consistent.

Further, it is worth noting that pushed Broker configuration is synchronized through the ECP Component Directory and thus it takes some time for changes to come into effect.

8.2. How to configure component filter

Components filtering provides Broker with the ability to reject messages coming from unauthorized components or to reject messages with unsupported message types. This can be done by configuring a list of allowed component codes or a list of allowed message types. It is possible to configure both lists, or only one of them.

The components list applies to both senders and receivers. When the component list is not configured, all components can communicate through the Broker.

When the message types list is not configured, then any message type is allowed.

Property	Description	Default value
ecp.broker.filter.components	List of ECP component codes that can communicate through Broker, separated by commas. The wild card symbol is supported. See section *Message type matching rules*.	N/A

ecp.broker.filter.types	List of message types that can be exchanged through Broker, separated by commas. The wild card symbol is supported. See section *Message type matching rules*.	N/A

8.3. How to configure NAT

Network address translation (NAT) is used for synchronization with the Component Directory and sending statistics to the Component Directory. For using NAT, at least one record of the NAT table must be defined. Each network address is compared to the NAT table to search a matching record with the same network code, IP address and port. If a match is found, the IP address and the port is translated.

Property	Description	Default value
ecp.natEnabled	This property enables (true) or disables (false) NAT.	false
ecp.natTable[i].network	Network code of the network address. If not provided, default code is used.	N/A
ecp.natTable[i].sourceIp	Source IP address to match. This property is mandatory.	N/A
ecp.natTable[i].destinationIp	Source IP will be translated to this IP address. This property is mandatory.	N/A
ecp.natTable[i].sourcePort	Source port to match. If not provided, matching will be based only on IP address.	N/A
ecp.natTable[i].destinationPort	Source port will be translated to this port. Must be defined when sourcePort is provided.	N/A

Where *i* represents an index of a record in the NAT table starting from 0.

8.4. Certificates renewal

ECP Broker supports only manual certificates renewal. Certificates must be updated before the expiration of the old ones.

1. *Go to the folder /opt/ecp-broker/reg with* ***broker-registration-tool.jar***
2. *Execute the commands mentioned below:*
   - *java -D"spring.config.location=/opt/ecp-broker/config/broker.properties" -jar broker-registration-tool.jar -m UPDATE_AUTH_CERTIFICATE*
     - Downloads the new Authentication certificate from Component Directory
   - *java -D"spring.config.location=/opt/ecp-broker/config/broker.properties" -jar broker-registration-tool.jar -m PUSH_CONFIGURATION*
     - After a successful renewal of the Authentication certificate, it is necessary to push the changes to the Component Directory
3. *Wait until all ECP components are synchronized to distribute the new Authentication certificate to all components*
4. *Perform a restart of ECP Broker*

New certificates will have Valid From time set to one hour in the past.

8.5. Artemis configuration

8.5.1. Enable HTTPS

It is possible to enable HTTPS for the Artemis Management Console. Open ***bootstrap.xml*** in */opt/ecp-broker/broker/etc* folder and find the *binding* element. Change *http://* to *https://* in the *uri* attribute. Then configure the following attributes:

- *keyStorePath – A path to the keystore file.*
- *keyStorePassword – A keystore password. Can be masked using ENC() syntax.*
- *sniHostCheck – Set it to false when the certificate in the keystore does not contain a host name in the Common Name or in the Subject Alternative Names. Default is true.*
- *includedTLSProtocols – TLSv1.3 is a recommended protocol.*
- *includedCipherSuites – Following ciphers are recommended: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256*

8.5.2. Enabling remote JMX access

Artemis supports monitoring by using JMX, which is enabled by default. It can be disabled by adding this line to */opt/ecp-broker/broker/etc/broker.xml*:

> *<jmx-management-enabled>false</jmx-management-enabled>*

By default, JMX is enabled for a local access only. For enabling the remote JMX access, follow the steps bellow:

1. *Edit **management.xml** file located in the folder /opt/ecp-broker/broker/etc/*
2. *Find and uncomment this row*
   > *<connector connector-port="1099"/>*
3. *To secure the connector by using SSL, add and configure these parameters to the connector element*
   > *secured – Boolean whether the connector is secured using SSL*
   > *key-store-path - The location of the keystore*
   > *key-store-password - The keystore password*
   > *trust-store-path - The location of the truststore*
   > *trust-store-password - The truststore password*
4. *Save the file.*
5. *Restart ECP Broker.*
6. *After the restart you can remotely connect to ECP Broker using JMX.*

8.5.3. Passwords encryption

Passwords in Artemis configuration files can be encrypted. Any encrypted password must be wrapped in *ENC()* to be treated by Artemis as an encrypted password. Artemis provides a script for the password encryption:

> */opt/ecp-broker/broker/bin/artemis mask '<password input>'*
> *An encrypted password will be printed. Copy all characters behind the "result:" and use them in a configuration file, wrapped in ENC()*

Encrypted passwords can be used in these configuration files in /opt/*ecp-broker/broker/etc*:

> *broker.xml*
> *login.config*
> *bootstrap.xml*
> *management.xml*

By default, Artemis uses a deprecated default password codec. Therefore, it is recommended to use a custom password codec. Please refer to the Artemis documentation for more details.

Passwords in **artemis-users.properties** cannot be encrypted, but they can be hashed (see the next chapter).

8.5.4. User management

Users that can access the Artemis Management Console or establish a connection to the AMQPS acceptor are defined in */opt/ecp-broker/broker/etc* **artemis-users.properties** file. For users accessing the console, it is possible to use the LDAP authentication instead (see the next chapter).

A new installation comes with one predefined *admin* user. It is strongly recommended to replace this user with your own or at least change the password. Users are stored in the following format:

> *username = password*

A new users can be added manually in the file. The Artemis script for creating a hashed password can be used:

> */opt/ecp-broker/broker/bin/artemis mask --hash '<password input>'*

> *A hashed password will be printed. Copy all characters behind the "result: 1024:" and use them in the* ***artemis-users.properties*** *file, wrapped in ENC()*

8.5.5. LDAP authentication

It is possible to enable LDAP (Lightweight Directory Access Protocol) authentication in the Artemis Management Console. With LDAP authentication enabled, **artemis-users.properties** file is then used only for the AMQP connections, and not for the console.

The LDAP server must be either the Active Directory server or a server supporting the standard LDAP protocol. A connection to a server can be made by LDAP or LDAPS (LDAP secured with TLS) protocol. LDAPS requires a truststore with a public certificate of the LDAP server. If LDAP server requires a client certificate authentication, a keystore with a client keypair is required.

Following steps must be done to enable LDAP authentication:

5. *Backup* ***bootstrap.xml and broker.xml*** *from /opt/ecp-broker/broker/etc folder.*
6. *In* ***bootstrap.xml****, rename domain in the jaas-security element from "activemq" to "activemq-properties".*

7. *Copy and replace login.config from /opt/ecp-broker/broker/etc/ldap or from /opt/ecp-broker/broker/etc/ldap/activeDirectory (when Active Directory is used) to /opt/ecp-broker/broker/etc.*
8. *Configure properties in **login.config**.*

The login module in ***login.config*** is a custom ECCo SP module, which extends the Artemis login module with a possibility to use LDAPS connection. It also changes the handling of the Active Directory bind user; Instead of configuring the bind user and password, values from the login form are used for the binding.

These properties are common for both Active Directory and standard LDAP:

Property	Description	Default value
initialContextFactory	The initial context factory used to connect to LDAP. It must always be set to the default value.	com.sun.jndi.ldap.LdapCtxFactory
connectionUrl	URL and a port number of the LDAP server. It must start with “ldap://” or “ldaps://”. If it starts with “ldaps://”, a connection will be secured with TLS. You can optionally qualify this URL by adding a forward slash `/`, followed by the DN (distinguished name) of a particular node in the directory tree.	ldaps://127.0.0.1:636
connectionProtocol	Change it to a blank string when LDAPS is not used.	ssl
connectionTimeout	A request will timeout when a connection to the LDAP server is not established within this time limit in milliseconds.	5000
readTimeout	A request will timeout when the LDAP server will not return a result within this time limit in milliseconds, after a connection has been established.	5000
authentication	Specifies the authentication method used when binding to the LDAP server. Can be set to *simple* (username and password) or *none* (anonymous).	simple
verifyHost	Disables hostname verification. Set it to *false* when certificates do not contain LDAP server hostname in the Common Name or Subject Alternative Names.	true
truststoreLocation	A path to the truststore file with a public certificate of the LDAP server.	/opt/ecp-broker/broker/

		etc/ldap-truststore.jks
truststorePassword	A truststore password.	password
keystoreLocation	A path to the keystore file with a client private keypair.	/opt/ecp-broker/broker/etc/ldap-keystore.jks
keystorePassword	A keystore password.	password
keyPassword	A password to a keypair in the keystore.	password
userSearchSubtree	Specify the search depth for user entries, relative to the node specified by *userBase*. *False* indicates it will try to match one of the child entries of the *userBase* node. *True* indicates it will try to match any entry belonging to the subtree of the *userBase* node.	false

This property can be changed for Active Directory:

Property	Description	Default value
connectionUsername	This property is not used with Active Directory, because the username from the login form is used as a bind user.	ignored
connectionPassword	This property is not used with Active Directory, because the password from the login form is used as a bind password.	ignored
userBase	Selects a particular subtree to search for user entries.	cn=users,dc=domain,dc=com
userSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *userBase*. The *{0}* substring is substituted by the username extracted from the incoming client credentials passed in the login form. Replace *domain.com* to match a domain from *userBase.*	(&(objectClass=user)(userPrincipalName={0}@domain.com))
roleName	Specifies the attribute type of the role entry that contains the name of the role/group (e.g. C, O, OU, CN etc.). If you omit this	CN

	option, the full DN (distinguished name) of the role is used.	
userRoleName	Specifies the name of the multi-valued attribute of the user entry that contains a list of role names for the user (where the role names are interpreted as group names by the broker's authorization plug-in). If you omit this option, no role names are extracted from the user entry.	memberof
ignorePartialResultException	Should be always set to *true*.	true

These properties must be changed for a standard LDAP:

Property	Description	Default value
connectionUsername	The DN of the user that opens the connection to the LDAP server.	uid=admin,ou=system
connectionPassword	The password that matches the DN from *connectionUsername*. In the LDAP server, the password is normally stored as a *userPassword* attribute in the corresponding directory entry.	password
userSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *userBase*. The *{0}* substring is substituted by the username extracted from the incoming client credentials passed in the login form.	(uid={0})
roleName	Specifies the attribute type of the role entry that contains the name of the role/group (e.g. C, O, OU, CN etc.). If you omit this option, the full DN (distinguished name) of the role is used.	cn
roleBase	If you want to store role data directly in the LDAP server, you can use a combination of role options (*roleBase*, *roleSearchMatching*, *roleSearchSubtree*, and *roleName*) as an alternative to (or in addition to) specifying the *userRoleName* option. This option selects a particular subtree DIT to search for role/group entries. The subtree is specified by a DN, which specifies the base node of	ou=groups,dc=domain,dc=com

	the subtree.	
roleSearchMatching	Specifies an LDAP search filter, which is applied to the subtree selected by *roleBase*. This works in a similar manner to the *userSearchMatching* option, except that it supports two substitution strings, as follows: {0} - substitutes the full DN of the matched user entry (that is, the result of the user search). For example, for the user *jdoe*, the substituted string could be *uid=jdoe,ou=User,ou=ActiveMQ,ou=system*. {1} - substitutes the received username. For example, *jdoe*. Leaving this property unset disables role searching, and the role information must come from *userRoleName*.	(uniqueMember ={0})
roleSearchSubtree	Specifies the search depth for role entries, relative to the node specified by *roleBase. False* indicates it will try to match one of the child entries of the *roleBase* node. *True* indicates it will try to match any entry belonging to the subtree of the *roleBase* node.	false

8.6. Plugins properties

ECP Broker uses several ECP plugins, each of them has configurable properties.

8.6.1. EcpPrometheusMetricsPlugin

EcpPrometheusMetricsPlugin configuration is in the */opt/ecp-broker/broker/etc/broker.xml* file, inside the *<metrics>* element.

Property	Description	Default value
ecpBrokerPropertiesLocation	A path to the **broker.properties** file. It should be changed only when the ECP Broker installation directory was moved from the default location.	/opt/ecp-broker/config/broker.properties

8.6.2. EcpBasePlugin

EcpBasePlugin configuration is in the */opt/ecp-broker/broker/etc/broker.xml* file, inside the *<broker-plugins>* element.

Property	Description	Default value
ecpBrokerPropertiesLocation	A path to the **broker.properties** file. It should be changed only when the ECP Broker installation directory was moved from the default location.	/opt/ecp-broker/config/broker.properties
ecpBrokerBeanConfigLocation	A path to the **ecp-config.xml** file. It should be changed only when the ECP Broker installation directory was moved from the default location.	/opt/ecp-broker/broker/etc/ecp-config.xml
ecpComponentType	ECP component type, which will be written in the logs. Default value should be kept.	ECP Broker
dlq	Name of the Dead Letter Queue, where the undeliverable messages will be sent.	DLQ
ecpCacheDisabled	Whether a cache holding authentication certificates should be disabled. When disabled, certificates are always loaded from the Component Directory content instead from a cache.	false
ecpCacheReloadInterval	How often is the cache holding the certificates refreshed. Interval is in milliseconds.	30000
clusterUser	Username used for a validation of a message sent from another node in Artemis cluster. Its value should be the same as the value of the *<cluster-user>* element in broker.xml. See the chapter *Plugins user credentials* for more details.	N/A

8.6.3. EcpAuthorizationPlugin

EcpAuthorizationPlugin configuration is in the */opt/ecp-broker/broker/etc/bootstrap.xml* file, inside the *<security-manager>* element.

Property	Description	Default value
ecpBrokerPropertiesLocation	A path to the **broker.properties** file. It should be changed only when the ECP Broker installation directory was moved from the default location.	/opt/ecp-broker/config/broker.properties
ecpCacheDisabled	Whether a cache holding authentication certificates should be disabled. When disabled, certificates are always loaded from the Component Directory content instead from a cache.	false
ecpCacheReloadInterval	How often is the cache holding the certificates refreshed. Interval is in milliseconds.	30000
clusterUser	Username used for a validation of a message sent from another node in Artemis cluster, management message, or Artemis CLI command request. When the ECP Broker is in Artemis cluster, the value should be the same as the value of the *<cluster-user>* element in broker.xml. See the chapter *Plugins user credentials* for more details.	N/A
clusterPassword	Password used for a validation of a message sent from another node in Artemis cluster, management message, or Artemis CLI command request. When the ECP Broker is in Artemis cluster, the value should be the same as the value of the *<cluster-user>* element in broker.xml. When encrypted, it must be wrapped inside *ENC(…)*. See the chapter *Plugins user credentials* for more details.	N/A
passwordCodec	An Artemis password codec used for the encryption of the *clusterPassword.* When not defined, a default Artemis	N/A

codec is used.

See the chapter *Plugins user credentials* for more details.

8.7. Plugins user credentials

ECP plugins in ECP Broker validate and authenticate messages and connections by loading an ECP component code from the authentication certificate used for a connection from ECP component to ECP Broker.

When a message is sent to ECP Broker from another ECP Broker node in the Artemis cluster (due to the load balancing or redistribution), when a management message is received (using Artemis CORE protocol), or when a request from Artemis CLI command is received, then the ECP certificate authentication will not work; There is no certificate sent in these cases, because they expect the user and password authentication instead.

To allow user and password authentication, it is possible to configure *clusterUser* and *clusterPassword* plugin properties. When defined, ECP Broker will try first to perform user and password authentication before the certificate authentication. See the *Plugins properties* chapter for details on how to configure these properties.

Only one user and password is allowed to be configured in the plugin properties. When ECP Broker is in Artemis cluster, then *clusterUser* and *clusterPassword* must use the same values as the *<cluster-user>* and *<cluster-password>* elements from the cluster configuration in the **broker.xml** file.

When a password in *clusterPassword* is encrypted, it must be written in the Artemis *ENC(…)* syntax. When a custom password codec was used, it must be configured in the *passwordCodec* plugin property. A custom key used for the encryption can be defined either in *artemis.default.sensitive.string.codec.key* Java property, or in *ARTEMIS_DEFAULT_SENSITIVE_STRING_CODEC_KEY* environment property.

8.8. Backup and restore data and configuration

This chapter describes how to backup Broker data and configuration.

Backup these files and folders containing Broker data and configuration. Copy backed up files to these paths for installation restore.

- */opt/ecp-broker/broker/etc*
- */opt/ecp-broker/broker/data*
- */opt/ecp-broker/config*

- */opt/ecp-broker/cd*

8.9. Required Broker configuration

The configuration file ***broker.properties*** comes with many configuration parameters (all of them are briefly described in the ***broker.properties*** file and also in this section). Some of the configuration parameters use the default value, but the following parameters must be configured for each installation:

8.9.1.1. Registration process properties

Parameter	Description
ecp.broker.contactPerson	Broker registration data
ecp.broker.contactEmail	Broker registration data
ecp.broker.contactPhone	Broker registration data
ecp.broker.organization	Broker registration data
ecp.broker.urls	Broker URL with pattern <protocol>://<host>:<port>. Broker protocol is typically amqps. Multiple values are separated by comma (for HA deployment). e.g. amqps://192.168.1.1:5671
ecp.broker.networks	Comma separated list of physical network names to which the URLs (ecp.broker.urls) belong. The number of networks must be equal to the number of URLs and the order must match because URL and network forms a pair. e.g. Internet
ecp.broker.code	ECP Broker component code
ecp.broker.filter.components	Define which components can communicate through broker. Multiple values of component codes separated by comma. Wildcard character * can be used to allow communication for all components. Default value is *
ecp.broker.filter.types	Define which message types are accepted by the broker. Multiple values are separated by comma Wildcard character * can be used to accept all message types. Default value is *

8.9.1.2. Broker component directory properties

Parameter	Description
ecp.directory.client.synchronization.homeComponentDirectoryPrimaryCode	Component code of home Component Directory
ecp.directory.client.synchronization.homeComponentDirectoryPrimaryUrl	URL of home Component Directory. CD URL with the following pattern: <protocol>://<host>:<port>/ECP_MODULE. CD protocol is typically https. (E.g. *https://127.0.0.1:8443/ECP_MODULE*)
ecp.directory.client.synchronization.directorySynchronizationInterval	Component Directory synchronization interval. CRON expression, default value is 0 */5 * * * *
ecp.directory.client.root	Customizable location of Component Directory data stored on Broker. Location must exist at Broker filesystem. e.g.: /opt/ecp-broker/cd

8.9.1.3. Broker keystores properties

Parameter	Description
ecp.keystore.location	Location of keystore with registration certificates. This keystore is provided by the Component Directory administrator and was listed in the prerequisites. After the Broker registration, an authentication keypair is added to this keystore. This keypair is also updated when the certificate renewal was made.
ecp.keystore.password	Registration keystore password (by default this is “password”)
ecp.authKeystore.location	Location of the keystore with authentication certificates (e.g. /opt/ecp-broker/authKeystore.jks). This location must point to an unexisting file as it will be used by ECP to save new keystore after a successful registration.
ecp.authKeystore.password	Authentication keystore password (e.g. password).

9. Monitoring

ECP provides four ways to check or monitor the status of the component.

- *A GUI interface suitable for the human users, who can check the basic health indicators on the dashboard and the messaging statistics pages.*
- *A JMX (Java Management Extension) interface suitable for monitoring systems like Nagios that can use this Java-specific technology to monitor the health of the component including low-level details such as the memory usage or the number of active threads.*
- *A Jolokia REST-style API suitable for machine users and monitoring systems that can use HTTP protocol to obtain any information available through the JMX. Jolokia functions as an HTTP bridge for JMX. Please refer to https://jolokia.org/ for more information.*
- *Prometheus metrics for Prometheus and Grafana monitoring systems for monitoring the health of the component including low-level details such as the memory usage or the number of active threads*

Since each ECP component (Endpoint, Broker, Component Directory) provides different functions, the metrics and information provided in monitoring are described separately for each component.

9.1. Endpoint

The endpoint exposes its monitoring metrics through a dashboard screen in the GUI and through the JMX and Jolokia REST interfaces. The endpoint provides information about its overall status and about the messaging processes.

The table below lists the metrics available through the monitoring.

Group	Name	Description
Connectivity	# connected brokers	Number of brokers to which the endpoint is connected.
	# brokers with broken connection	Number of brokers that have connectivity issues.
Messaging	# sent messages	Number of messages sent through this endpoint by business applications since it was installed.
	# received messages	Number of messages received through this endpoint by BAs since it

		was installed.
	# waiting to deliver messages	Number of messages waiting to be delivered to the receiver endpoint.
	# waiting to receive messages	Number of messages waiting on the endpoint to be received by a BA.
	# messages in store	Number of messages in the message store.
	# live messages in store	Number of messages in the message store that are not in the final state.
	# messages to be archived	Number of messages in the message store that are in the final state and waiting to be archived.
Component Directory	Endpoint status	Status of the endpoint's registration at home directory.
	Certificates status	Information if the certificate set is valid and complete.
	Closest certificate expiration date	Closest date when a certificate expires.
	Message paths status	Information if all message paths are valid.
	# invalid message paths	Number of invalid message paths.
	Synchronization status	Status of the synchronization with a component directory.
	Version check	Information if there is a newer version installed on the home component directory.

If the ECP is configured to use an external database, which is required only for an HA deployment, the monitoring should include the database health as well. This topic is out of ECP scope.

9.2. Broker and Internal Broker

The monitoring of the message broker is limited to the API provided by the AMQP broker, i.e. Artemis. It provides an administration web console with a GUI to the broker administrator and a JMX API as an interface to external monitoring tools. There is no custom interface implemented within ECP. Any tool with JMX support can be used to monitor the broker.

JMX is enabled by default in Artemis. More details about the JMX configuration can be found in https://activemq.apache.org/components/artemis/documentation/2.31.2/management.html#management-via-jmx

9.3. Component Directory

The Component Directory has the same way of providing the monitoring features as the Endpoint. Only the set of provided information and metrics is different, since the Component Directory does not participate in the messaging and its main purpose is to manage all security information.

All the monitoring information available is listed in the table below.

Group	Name	Description
Components	# of waiting approvals	Number of requests waiting for registration approval.
	# of active components	Number of components in active state registered at the component directory.
	# of expired components	Number of components with expired certificates.
	Synchronization status	Status of the synchronization with other component directories.
Certificates	Certificates status	Information if the certificate set is valid and complete.
	Closest certificate expiration date	Closest date when any certificate expires.

If the ECP is configured to use an external database, which is required only for the HA deployment, the monitoring should include the database health as well. This topic is out of ECP scope.

9.4. Info Endpoint

ECP exposes by default the Info Endpoint with information about the ECP application. Info Endpoint is accessible on **/ECP_MODULE/actuator/info* URL with HTTPS port. If it is not accessible, verify that there is the **management.endpoints.web.exposure.include** property in the ecp/ecp-directory.properties configuration file with **info** in the list of values.

Info Endpoint provides this information:

- *System Name – ECP Component Directory or ECP Endpoint*
- *System Description – ECP application description*
- *System Version – ECP version*
- *Component Code – ECP component code*
- *Cluster Node Code – ECP node identifier in the cluster*
- *Java Version – Java version installed on a host server*

9.5. Health Endpoint

ECP exposes by default the Health Endpoint with information about the ECP application status. Health Endpoint is accessible on **/ECP_MODULE/actuator/health* URL with HTTPS port. If it is not accessible, verify that there is the **management.endpoints.web.exposure.include** property in the ecp/ecp-directory.properties configuration file with **health** in the list of values.

Health Endpoint provides *status* parameter, which could be either *UP* or *DOWN*. The status depends on the several default health checks (database, disk space, jms, ping) and one custom check, which use a set of ECP metrics and conditions to determine the status.

Component Directory metrics check:

Metric name	Condition for DOWN	Description
synchronizationStatus	False	Some of the synchronization with other Component Directories was not successful.
certificateStatus	False	Some certificate is expired or missing.
numberOfActiveComponents	< 0	Number of registered active component is less than a condition value. It is set to zero by default, to prevent DOWN status when there is not yet any registered component.
numberOfExpiredComponents	> 0	Number of components with expired certificates is greater than a condition value.
isReloading	True	Whether the Component Directory is currently reloading.
countOfComponentsWithNotRenewedCertificates	> 0	Number of components which certificates are going to expire is greater than a condition value.
countOfEndpointsWithoutValidMessagePath	> 0	Number of Endpoints without valid message path is greater than a condition value.
countOfNotSynchronizedComponents	> 0	Number of components not synchronized in last 10 minutes is greater than a condition value.
countOfNotSynchronizedDirectories	> 0	Number of directories not synchronized in last 15 minutes is greater than a condition value.

Endpoint metrics check:

Metric name	Condition for DOWN	Description
internalBrokerConnected	Not connected	Whether a connection to Internal Broker was established.
connectedBrokers	< 1	Number of connected ECP Brokers is less than a condition value.
waitingToDeliver	> 50	Number of messages waiting to be delivered is greater than a condition value.
waitingToReceive	> 50	Number of messages waiting to be received is greater than a condition value.
isReloading	True	Whether the Endpoint is currently reloading.
synchronizationStatus	False	The Component Directories synchronization was not successful.
certificateStatus	False	Some certificate is expired or missing.
invalidMessagePath	> 0	Number of invalid message paths is greater than a condition value.
withoutValidMessagePath	True	Whether the Endpoint has no valid message path.

It is possible to define a custom threshold number for each number metric by using this property:

Property	Description
ecp. healthThreshold.<metric name>	Threshold number for a condition check for a given metric. For example: ecp.healthThreshold. numberOfActiveComponents=10

It is possible to exclude any metric from the Health Endpoint by using this property:

Property	Description
ecp.healthExclude	Coma-separated list of metric names to exclude from the Health Endpoint, for example: ecp.healthExclude= certificateStatus,isReloading

9.6. Readiness Endpoint

ECP exposes by default the Readiness Endpoint with information whether the ECP application meets all the necessary conditions to be considered ready. Readiness Endpoint is accessible on **/ECP_MODULE/actuator/readiness* URL with HTTPS port. If it is not accessible, verify that there is the **management.endpoints.web.exposure.include** property in the ecp/ecp-directory.properties configuration file with **readiness** in the list of values. When Readiness Endpoint is enabled, Readiness tile is also displayed in ECP Dashboard.

When ECP component is ready, *status* parameter with *READY* value is returned. If ECP component is not ready, then it returns *status* with *NOT READY* value and *description* parameter with all reasons why the component is not ready.

These conditions are checked for ECP Component Directory:

- *Component Directory is registered (authentication certificate exists in the database)*
- *Authentication certificate is not expired*
- *Component Directory is not reloading a configuration*

These conditions are checked for ECP Endpoint:

- *Endpoint is in ACTIVE state*
- *Authentication certificate is not expired*
- *Endpoint is not reloading a configuration*
- *Endpoint is connected to an internal broker*
- *Endpoint is synchronized with Component Directory*
- *There is at least one valid message path*

9.7. Prometheus Monitoring

ECP supports monitoring by Prometheus system.

9.7.1. Component Directory and Endpoint

Prometheus metrics are disabled by default. To enable it, add **prometheus** to the list in the **management.endpoints.web.exposure.include** property, or create the property if it not exists in the ecp/ecp-directory.properties configuration file. Then add this property into ecp/ecp-directory.properties configuration file and restart ECP component:

> *endpoints.prometheus.sensitive=false*

Prometheus metrics are accessible on **/ECP_MODULE/actuator/prometheus* URL with HTTPS port. To connect ECP metrics with Prometheus monitoring system, it is necessary to edit Prometheus configuration file – **prometheus.yml**. Here, create a new job in *scrape_configs* section like this example:

- job_name: 'ecp'

metrics_path: '/actuator/*prometheus'*

scrape_interval: 5s

static_configs:

- targets: ['localhost:8443']

basic_auth:

username: admin

password: password

You can change *job_name* and *scrape_interval* values as required. Change *targets* value to match the IP address of the ECP application server and ECP HTTP or HTTPS port. Section *basic_auth* is mandatory only if ECP user authentication is enabled, *username* and *password* values must match one of the ECP user. If you use HTTPS for metrics access, add these properties to the job configuration:

scheme: https

tls_config:

insecure_skip_verify: true

The ECP version is provided in four separated number metrics. Values from these metric can be merged to get the ECP version (*main.major.minor.build*).

ECP Component Directory provides these custom metrics:

> *ecp_directory_number_of_pending_registrations - Number of requests waiting for registration approval*

> *ecp_directory_number_of_active_components - Number of components in active state registered at the Component Directory*

> *ecp_directory_number_of_expired_components - Number of components with expired certificates*

> *ecp_directory_synchronization_status - Status of the synchronization with other Component Directories*

> *ecp_directory_certificates_status - Information if the certificate set is valid and complete*

> *ecp_directory_reloading - Information if the Component Directory is reloading*

- *ecp_directory_number_of_certificates_expire_soon - Number of components with certificates expiring in 30 days (by default, value is taken from ecp.automaticUpdate.dayBefore) or less*
- *ecp_directory_number_of_endpoints_without_valid_message_path - Number of Endpoints without valid message path*
- *ecp_directory_number_of_not_sync_components - Number of components which were not synchronized in last 10 minutes (by default, value can be changed in ecp.metricsSyncThreshold property)*
- *ecp_directory_number_of_not_sync_directories - Number of component directories which were not synchronized in last 15 minutes*
- *ecp_directory_certificate_closest_expiration – Number of days until the closest certificate expiration*
- *ecp_directory_job_success{job=<job name>} – Whether the last job run was successful (1.0)*
- *ecp_directory_job_start{job=<job name>} – Time of the start of the last job run*
- *ecp_directory_job_end{job=<job name>} – Time of the end of the last job run*
- *ecp_directory_version – ECP main version ("**4**".14.0.2019)*
- *ecp_directory_major_version – ECP major version (4."**14**".0.2019)*
- *ecp_directory_minor_version – ECP minor version (4.14."**0**".2019)*
- *ecp_directory_build_version – ECP build number (4.14.0."**2019**")*

ECP Endpoint provides these custom metrics:

- *ecp_endpoint_certificates_status - Information if the certificate set is valid and complete*
- *ecp_endpoint_is_valid_message_path - Information if all message paths are valid*
- *ecp_endpoint_number_of_invalid_message_paths - Number of invalid message paths*
- *ecp_endpoint_synchronization_status - Status of the synchronization with a Component Directory*
- *ecp_endpoint_version_check - Information if there is a newer version installed on the home Component Directory*
- *ecp_endpoint_connected_brokers - Number of brokers that the Endpoint is connected to*
- *ecp_endpoint_connected_broken_brokers - Number of brokers that have connectivity issues*
- *ecp_endpoint_sent_messages - Number of messages sent through this Endpoint by BAs since it was installed*

- *ecp_endpoint_received_messages - Number of messages received through this Endpoint by BAs since it was installed."*
- *ecp_endpoint_waiting_to_deliver_messages - Number of messages waiting to be delivered to the receiver Endpoint*
- *ecp_endpoint_waiting_to_receive_messages - Number of messages waiting on the Endpoint to be received by a BA*
- *ecp_endpoint_messages_in_store - Number of messages in the message store*
- *ecp_endpoint_live_messages_in_store - Number of messages in the Message Store that are not in the final state*
- *ecp_endpoint_messages_to_be_archived - Number of messages in the Message Store that are in the final state and are waiting to be archived*
- *ecp_endpoint_reloading - Information if the Endpoint is reloading*
- *ecp_endpoint_certificate_expire_soon - Information if there is any preferred certificate that expires in 30 days (by default, value is taken from ecp.automaticUpdate.dayBefore) or less*
- *ecp_endpoint_is_without_valid_path - Information if there is a component which has 1 or more invalid paths and 0 valid paths*
- *ecp_endpoint_certificate_closest_expiration – Number of days until the closest certificate expiration*
- *ecp_endpoint_job_success{job=<job name>} – Whether the last job run was successful (1.0)*
- *ecp_endpoint_job_start{job=<job name>} – Time of the start of the last job run*
- *ecp_endpoint_job_end{job=<job name>} – Time of the end of the last job run*
- *ecp_endpoint_version – ECP main version ("**4**".14.0.2019)*
- *ecp_endpoint_major_version – ECP major version (4."**14**".0.2019)*
- *ecp_endpoint_minor_version – ECP minor version (4.14."**0**".2019)*
- *ecp_endpoint_build_version – ECP build number (4.14.0."**2019**")*

9.7.2. Broker and Internal Broker

Prometheus metrics are accessible on */metrics* URL with Artemis Management Console port. Note that the authentication is not yet supported in the Artemis Prometheus metrics plugin, so the metrics will not be protected when a remote access is enabled.

To connect ECP metrics with Prometheus monitoring system, it is necessary to edit Prometheus configuration file – **prometheus.yml**. Here, create a new job in *scrape_configs* section like this example:

- job_name: 'ecp'

metrics_path: '/metrics'

scrape_interval: 5s

static_configs:

- targets: ['localhost:8161']

You can change *job_name* and *scrape_interval* values as required. Change *targets* value to match the IP address of the application server and the Artemis Management Console port. If you use HTTPS for the Web Console, add these properties to the job configuration:

scheme: https

tls_config:

insecure_skip_verify: true

Please note that the Artemis Prometheus metrics plugin, which exposes the metrics at */metrics* URL, does not support the authentication. If you wish to enable a remote access to the console without exposing the metrics, it is possible to disable the metrics in **bootstrap.xml**:

- *In the binding element, find and remove this line:*
  - *<app url="metrics" war="metrics.war"/>*

ECP Broker provides these custom metrics:

- *ecp_broker_version – ECP main version ("**4**".14.0.2019)*
- *ecp_broker_major_version – ECP major version (4."**14**".0.2019)*
- *ecp_broker_minor_version – ECP minor version (4.14."**0**".2019)*
- *ecp_broker_build_version – ECP build number (4.14.0."**2019**")*
- *ecp_broker_daysToClosestCertificateExpiration - Number of days til the closest certificate expiration in the local keystore*

The ECP version is provided in four separated number metrics. Values from these metric can be merged to get the ECP version (*main.major.minor.build*).

Internal Broker does not provide any ECP custom metrics.

10. Figure Content

Figure 1: New Path 37
Figure 2: New Central Message Path 89